

Raport Final

Petrea Bogdan-Vasile 342C3

INCHIRIERI BICICLETE

Acest raport detaliaza procesul de analiza, preprocesare si antrenare a unor modele pentru a putea gasi cele mai bune configurari pentru a putea lucra cu un set de date care evidentiaza inchirierea de biciclete. Obiectivul final este predictia numarului total de biciclete inchiriate pe ora, folosind datele puse la dispozitie, acestea avand informatii legate de conditiile meteo, si calendaristice.

Partea 1: Analiza Exploratorie a Datelor (EDA)

Prima parte legata de cerinta vizeaza analiza setului de date pe care l-am primit. Astfel, aceasta parte ne ofera un overview legat de datele cu care vom lucra, structura acestora, cum sunt impartite, cum se vor influenta attributele intre ele; putand astfel sa ne decidem cum vom procesa datele mai departe.

1. Prima analiza vizeaza structura setului si prezenta valorilor lipsa. Astfel putem intelege tipurile de date cu care vom lucra, daca exista valori lipsa, ce variabile ne dorim sa le transformam si cum. Setul de date contine 6878 inregistrari, avand 19 coloane (attribute) dupa modificarile facute.

#	Column	Non-Null Count	Dtype
0	sezon	6878 non-null	int64
1	sarbatoare	6878 non-null	int64
2	zi_lucratoare	6878 non-null	int64
3	vreme	6878 non-null	int64
4	temperatura	6878 non-null	float64
5	temperatura_resimtita	6878 non-null	float64
6	umiditate	6878 non-null	int64
7	viteza_vant	6878 non-null	float64
8	ocasionali	6878 non-null	int64
9	inregistrati	6878 non-null	int64
10	total	6878 non-null	int64
11	ora	6878 non-null	int32
12	zi_saptamana	6878 non-null	int32
13	luna	6878 non-null	int32
14	an	6878 non-null	int32
15	saptamana	6878 non-null	int64
16	is_weekend	6878 non-null	int64
17	sin_ora	6878 non-null	float64
18	cos_ora	6878 non-null	float64

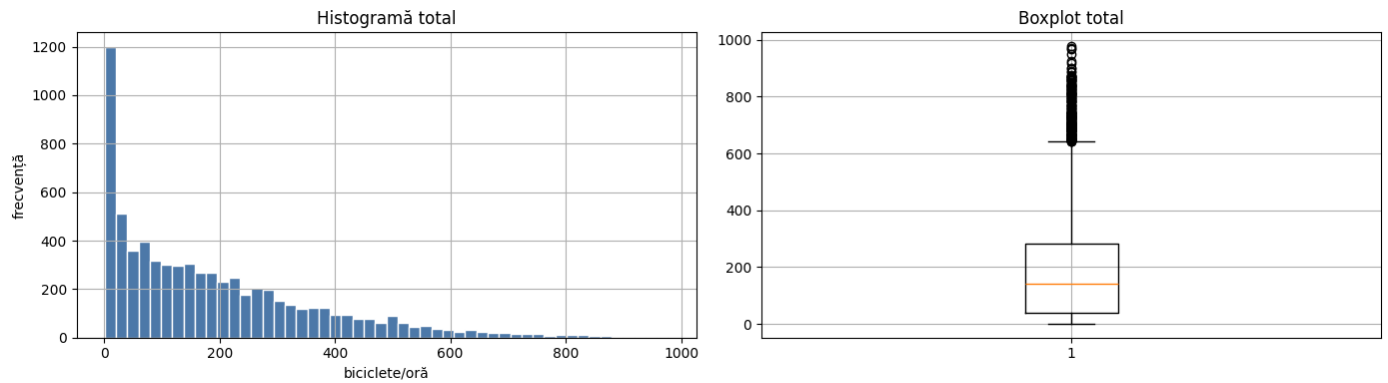
Prima data am sppart atributul **data_ora** in mai multe atribute, cum ar fi: ora, zi_luna, luna, an, saptamana, is_weekend, si in doua functii matematice pentru modelarea orelor: sin_ora, respectiv cos_ora (Acest lucru ne ajuta pentru a putea modela mai usor relatia dintre ore. Daca am fi folosit One-Hot Encoding pentru aceasta parte, intre ora 23 si 00 ar fi fost o diferenta foarte mare, dar functiile matematice ne vor incadra valorile in intervalul [-1,1], pastrand aceste doua ore apropiate).

Mai departe, am afisat primele 5 randuri din dataset, tipurile datelor, dar si cateva date despre acestea, cum ar fi numarul de valori lipsa pentru fiecare atribut, media, deviatia standard, min, max, si quantilele de 25, 50, respectiv 75.

Sezon	Sarbatoare	Zi Lucratoare	Vreme	Temperatura	Temperatura Resimtita	Umiditate	Viteza Vant	Ocasionali	Inregistrati	Total	Ora	Zi Sap
count	6878	6878	6878	6878	6878	6878	6878	6878	6878	6878	6878	6878
mean	2.505670	0.031259	0.676	1.418436	20.077839	23.505789	61.990259	12.576206	35.356208	153.511	188.867	11.540
std	1.116669	0.174030	0.468	0.642373	8.168544	8.869723	19.827335	8.248823	49.064414	150.509	179.669	6.9150
min	1.000000	0.000000	0.000	1.000000	0.820000	0.760000	0.000000	0.000000	0.000000	1.000	0.000	0.000
25%	2.000000	0.000000	0.000	1.000000	13.120000	15.910000	46.000000	7.001500	4.000000	35.000	41.000	6.000
50%	3.000000	0.000000	1.000	1.000000	19.680000	23.485000	62.000000	11.001400	16.000000	115.000	141.000	12.000
75%	4.000000	0.000000	1.000	2.000000	27.060000	31.060000	78.000000	16.997900	48.000000	220.000	282.000	18.000
max	4.000000	1.000000	1.000	4.000000	41.000000	45.455000	100.000000	56.996900	362.000000	886.000	977.000	23.000

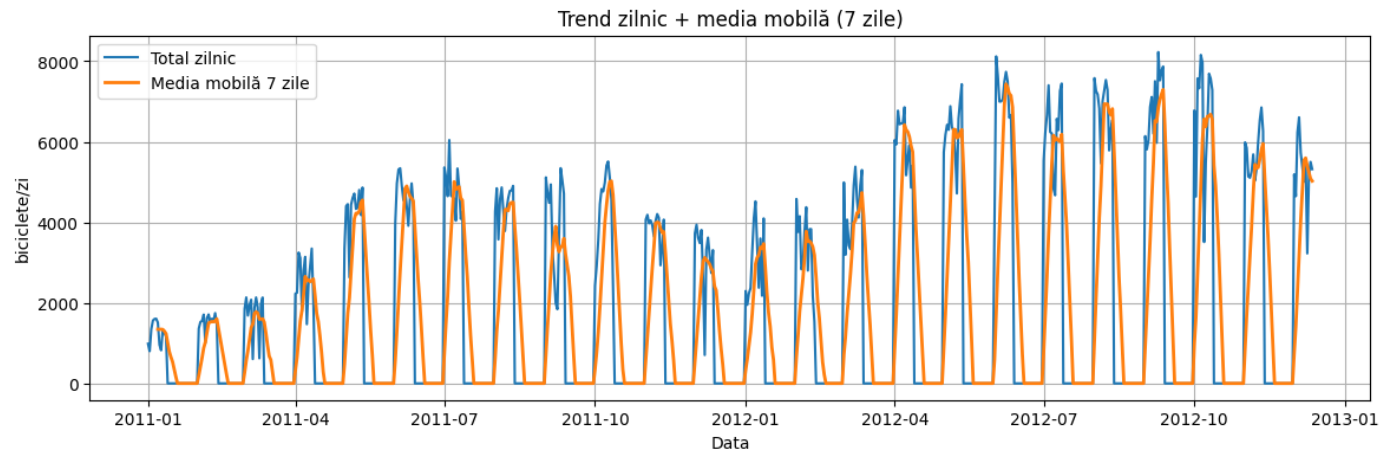
2. Distribuția Variabilei Target `total`

Aceasta analiza ne ajuta sa ne formam o imagine a cum sunt distribuite valorile in variabila target pentru setul de antrenare, pentru a ne putea forma o imagine, daca avem nevoie de modele care sa fie robuste la outlieri. Pentru aceasta parte am folosit o histograma si boxplot-ul, care ne indica prezenta outlierilor (valoarea 0 fiind foarte des intalnita, sau valori apropiate, lucru care logic este corect, existand orele de noapte cand nu se inchiriaza biciclete).

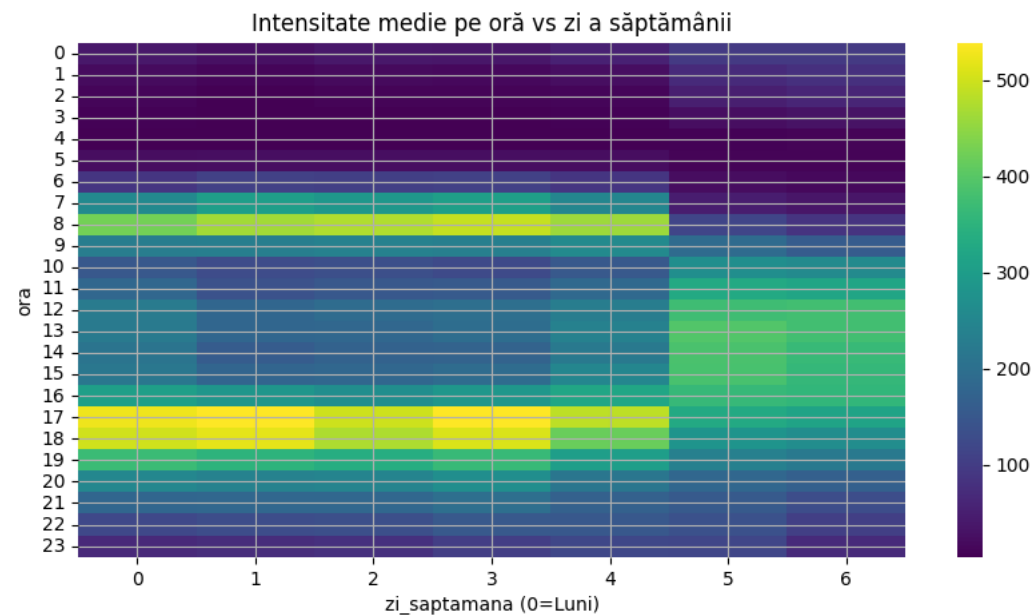


3. Serie de timp a datelor

Aceasta analiza poate fi una dintre cele mai utile, ajutandu-ne sa reperam modele ciclice in datele noastre, sa intelegem cum se modifica trendurile, pentru a ne putea astepta la anumite rezultate in functie de timp. Din graficele rezultate putem observa foarte clar ca exista o tendinta de crestere generala (toate varfurile se deplaseaza in sus fata de anul precedent in aceeasi perioada), dar si trenduri sezoniere, cum ar fi ca iarna, numarul de biciclete inchiriate scade, iar in sezonurile calde incepe sa creasca.

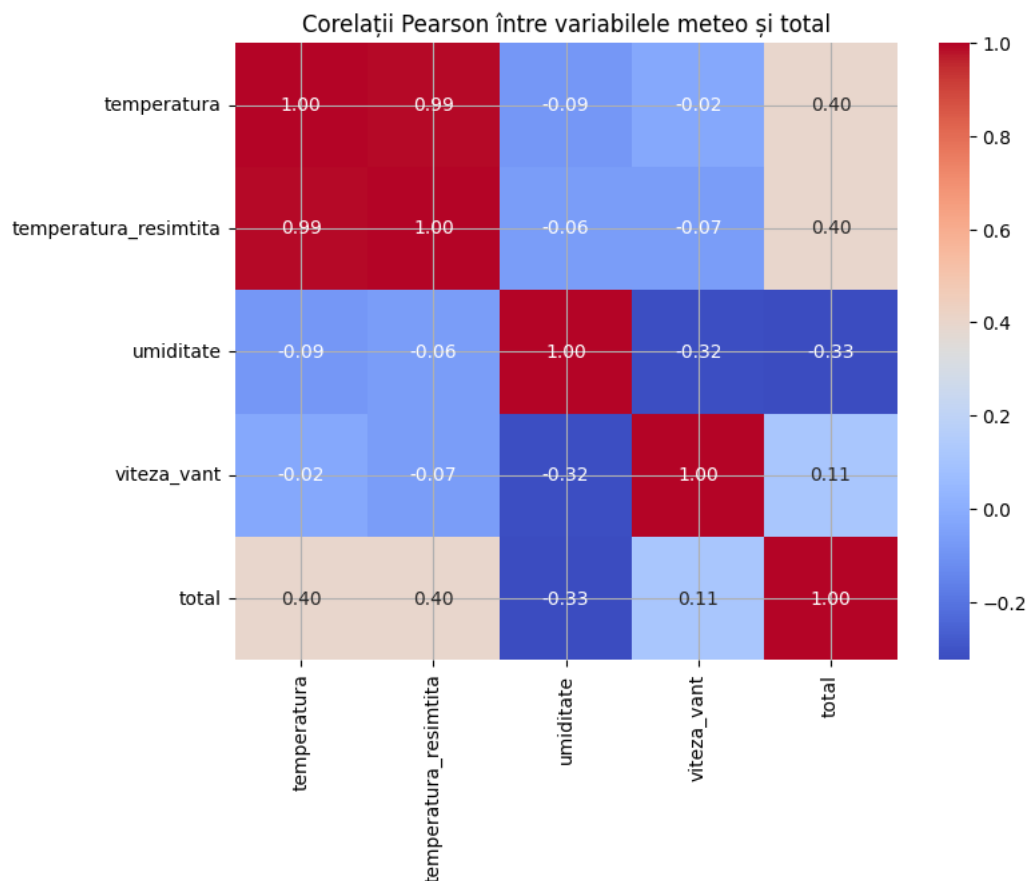


In plus, la nivel de saptamana, se poate observa un pattern, care ne sugereaza ca in zilele lucratoare (de luni pana vineri), la orele de varf, cum ar fi 8.00 si undeva in jur de ora 18.00, numarul de inchirieri creste brusc fata de alte ore. Comparativ, in weekend cresterea este mult mai lina, lucru observabil din graficele de mai jos.



4. Matricea de corelatie

Aceasta analiza ne ajuta sa ne dam seama cat de corelate sunt atributurile intre ele (adica, cat de mult furnizeaza aceleasi informatii), lucru util pentru a putea decide care dintre attribute sunt irelevante, neintroducand informatii suplimentare. Din matricea de corelatie, putem observa foarte usor ca `temperatura` si `temperatura_resimtita` sunt puternic corelate, lucru care ne indica ca putem sa renuntam la una dintre acestea.



Partea 2: Preprocesare (Extragerea, Standardizarea, Selecția de Atribute și Imputarea Valorilor Lipsă)

Pentru această parte ne bazăm pe concluziile din analiza exploratorie. Din aceasta am observat că targetul nostru, total, reprezintă suma a două atribute, înregistrați și ocazionali. Dacă am păstra aceste două coloane ca trăsături de intrare, modelul ar avea practic acces direct la componentele targetului și s-ar produce o scurgere de informație. Pentru a evita această problemă, am decis să eliminăm înregistrați și ocazionali din lista de atribute explicative și să păstrăm doar total ca variabilă țintă.

Setul de date a fost împărțit în două subseturi: 80% pentru antrenare (train) și 20% pentru validare (valid), astfel încât să putem evalua obiectiv performanța modelelor pe date nevăzute în timpul antrenării.

1. Extragerea de atribute

Pornind de la coloana temporală data_ora, am extras mai multe atribute care surprind structura calendaristică a fenomenului: ora, zi_saptamana, luna, an, saptamana și is_weekend. Aceste variabile ne permit să modelăm sezonabilitatea zilnică și săptămânală observată în analiza exploratorie.

Pentru a trata corect natura ciclică a orei, am transformat ora în două trăsături noi, sin_ora și cos_ora, folosind funcțiile trigonometrice sinus și cosinus. Această reprezentare păstrează faptul că ora 23 și ora 0 sunt "aproape" una de cealaltă, lucru care nu ar fi vizibil într-o codificare clasică one-hot sau dacă am trata ora doar ca un număr întreg obișnuit.

2. Transformări într-un pipeline

Pentru a organiza preprocesarea într-un mod clar și reproductibil, am definit un pipeline în care am împărțit trăsăturile în trei grupuri:

- Atribute numerice: temperatura, temperatura_resimtită, umiditate, viteză_vant, ora, zi_saptamana, luna, an, saptamana, sin_ora, cos_ora, is_weekend.

Asupra acestor variabile aplicăm mai întâi imputarea valorilor lipsă (unde este cazul) și apoi standardizarea. Standardizarea aduce fiecare atribut numeric la o scală comparabilă (valoare medie ≈ 0 și deviație standard ≈ 1), ceea ce împiedică dominația atributelor cu magnitudini mari (de exemplu, temperatură față de umiditate) și este esențială pentru algoritmi sensibili la scară, cum ar fi SVR.

- Atribute categorice: sezon, sarbatoare, zi_lucratoare.

Pentru acestea folosim One Hot Encoding. De exemplu, variabila sezon, care poate lua patru valori, este transformată în patru coloane binare, câte una pentru fiecare sezon. Astfel, modelul poate învăța separat efectul fiecărei categorii, fără a introduce o ordine artificială între ele.

- Atribute ordinale: vreme.

Variabila vreme are o interpretare ordonată (de la condiții meteo favorabile la condiții mai nefavorabile), astfel că este tratată ca un atribut ordinal, păstrându-se ordinea naturală a valorilor sale.

În plus, temperatura a fost discretizată în intervale (binuri), obținându-se o variabilă derivată care surprinde mai bine relația potențial neliniară dintre temperatură și numărul de biciclete închiriate. Gruparea valorilor de temperatură în binuri ajută modelul să distingă, de exemplu, între „foarte rece”, „plăcut” și „foarte cald”, fără a presupune că efectul crește liniar cu fiecare grad în parte.

3. Imputarea valorilor lipsă

Deși în setul de date inițial nu apar explicit valori lipsă, am inclus în pipeline și un pas de imputare pentru a face soluția robustă la posibile date viitoare incomplete sau la alte scenarii de utilizare. Pentru variabilele numerice folosim imputarea cu mediana, iar pentru cele categorice imputarea cu valoarea cea mai frecventă. În plus, imputarea este aplicată și înainte de discretizarea temperaturii, pentru a evita problemele cauzate de eventuale valori lipsă în acest câmp.

4. Selecția de atribute

După aplicarea întregului pipeline de preprocesare, am obținut în total 26 de trăsături. Pentru a verifica dacă toate contribuie semnificativ la predicție, am aplicat două metode de selecție a atributelor:

- SelectKBest, bazat pe informație mutuală între fiecare trăsătură și target.
- SelectFromModel, bazat pe un model Lasso antrenat cu diverși hiperparametri alpha, care penalizează trăsăturile mai puțin relevante.

Pentru fiecare configurație de selecție am antrenat un model de referință (Ridge) și am evaluat performanța pe setul de validare. Rezultatul a arătat că putem reduce numărul de trăsături de la 26 la 20 fără o degradare semnificativă a performanței (Valid_RMSE rămâne foarte apropiat de cel obținut cu toate trăsăturile). Astfel, modelul este mai simplu și mai ușor de interpretat, iar acest lucru sugerează că doar un subset al atributelor inițiale aduce cu adevărat informație utilă pentru predicția numărului de biciclete închiriate.

Din analiza matricei de corelație și a selecției de atribute reiese că cele mai predictive trăsături sunt cele legate de timp (ora, zi_saptamana, is_weekend, sezon), împreună cu variabilele meteo (temperatura, temperatura_resimtită, vreme, parțial umiditate). Acestea apar fie cu corelații mai ridicate cu targetul, fie sunt păstrate constant de metodele de selecție (SelectKBest, Lasso). Practic, modelul se bazează în principal pe combinația dintre momentul din zi / ziua din săptămână și condițiile meteo pentru a prezice numărul de biciclete închiriate, în timp ce alte trăsături au un rol secundar sau redundant.

config	n_features	Valid_RMSE
SelectKBest(k=20)	20	163.248372
SelectFromModel Lasso $\alpha=0.7$	10	163.750795
SelectFromModel Lasso $\alpha=0.3$	12	163.793976
SelectFromModel Lasso $\alpha=1.0$	9	163.805852
SelectFromModel Lasso $\alpha=0.05$	10	164.033366
SelectFromModel Lasso $\alpha=0.01$	10	164.033366
SelectFromModel Lasso $\alpha=0.001$	10	164.033366
SelectFromModel Lasso $\alpha=0.1$	12	164.320472
SelectKBest(k=13)	13	164.335612

Pe baza rezultatelor din tabel, observăm că cea mai bună configurație este SelectKBest(k=20), cu un Valid_RMSE ≈ 163.25 , ușor mai bun decât toate variantele SelectFromModel cu Lasso.

Pe baza rezultatelor din tabel, observăm că cea mai bună configurație este SelectKBest(k=20), cu un Valid_RMSE ≈ 163.25 , ușor mai bun decât toate variantele SelectFromModel cu Lasso.

În contextul celor două metode:

La SelectKBest, parametrul k reprezintă numărul de trăsături păstrate după selecție. În cazul nostru, k=20 înseamnă că, din toate cele 26 de trăsături generate de pipeline, metoda păstrează cele 20 considerate cele mai relevante (după informația mutuală cu targetul).

La SelectFromModel cu Lasso, parametrul alpha controlează forța regularizării L1:

- valori mai mari ale lui alpha (ex. 1.0, 0.7) înseamnă penalizare mai puternică, modelul "taie" mai agresiv coeficienții și tinde să păstreze mai puține trăsături (9–10 în tabel);
- valori mai mici ale lui alpha (ex. 0.1, 0.05, 0.01, 0.001) duc la o penalizare mai slabă, permițând mai multe trăsături active (10–12 în tabel), dar cu un risc puțin mai mare de suprapunere/complexitate.

Deși diferențele de RMSE între configurații sunt destul de mici (toate rezultatele sunt în intervalul ~ 163.2 – 164.3), SelectKBest(k=20) reușește să atingă cel mai bun compromis între performanță și dimensiunea setului de trăsături: reduce dimensionalitatea (de la 26 la 20 de feature-uri) fără să piardă informație relevantă și, în același timp, depășește ușor, ca eroare de validare, toate variantele bazate pe Lasso. Acest lucru sugerează că păstrarea unui număr moderat de trăsături (20) este suficientă pentru a obține performanță foarte bună, iar trunchierea mai agresivă la 9–12 trăsături aduce doar un mic compromis în RMSE.

Partea 3: Modele si Evaluare

Toate cele 6 modele au folosit RandomizedSearchCV pentru cautarea hiper parametrilor, iar TimerSeriesSplit (5 folduri) pentru Cross Validation. Toate metodele folosesc același pipeline, preprocesare, urmat de rularea efectivă a algoritmului, pe diverse combinații de hiper parametrii pentru a găsi cea mai bună combinație.

1. Linear Regression

Pentru acest algoritm am testat 3 variante:

- varianta standard;
- fără intercept (forțează modelul să treacă prin origine);
- cu coeficienți doar pozitivi (toți coeficienții sunt constrânși să fie ≥ 0).

Cea mai bună configurație a fost cea **fără intercept**. Cu toate acestea, valoarea obținută pe setul de validare ($MSE \approx 27\,927.8$) indică faptul că relația dintre trăsături și target este puternic neliniară, iar un model liniar simplu nu reușește să o surprindă corespunzător. În consecință, Linear Regression oferă predicții cu scoruri semnificativ mai slabe comparativ cu modelele pe arbori, fiind mai degrabă un baseline decât un candidat serios pentru modelul final.

2. SVR (Support Vector Regressor)

Hiper parametrii cautati:

- kernel = controleaza tipul relatiei pe care o poate invata modelul
 - linear = relatie aproximativ liniara intre trasaturi si target
 - rbf = permite relatii neliniare complexe, adaptate local
- C = termen de regularizare
 - c mic => model mai simpli, cu penalizare mai mare pentru erorile mari
 - C mare => model mai flexibil, risc pentru overfit
- epsilon = definește "tubul" în jurul curbei de regresie în care erorile nu sunt penalizate; valori mai mari de epsilon fac modelul mai tolerant la deviațiile mici și tind să producă o funcție de regresie mai netedă
- gamma (pentru kernel RBF) = controleaza cat de "locala" este influenta fiecarui punct de antrenare
 - gamma mic => functie mai neteda, cu influenta pe o zona mai mare
 - gamma mare => model foarte sensibil la punctele individuale (risc de overfit)

Cea mai buna configuratie gasita pentru SVR foloseste un **kernel RBF**, cu **C mare(100)**, **valoare moderata pentru gamma (0.01)** si un **epsilon de 0.3**, ceea ce permite modelului sa surprinda parti neliniare din date. Chiar și așa, performanța pe setul de validare ($MSE \approx 29656$, $RMSE \approx 172$) rămâne mai slabă decât în cazul modelelor pe arbori.

3. Random Forest Regressor

Hiper parametrii cautati:

- n_estimators = numarul de arbori din padure
 - valori mai mari => model mai stabil, dar timp de antrenare mai mare
 - valori mai mici => model mai rapid, dar cu varianta mai mare a predictiilor
- max_depth = adancimea maxima a fiecarui arbore
 - max_depth mic => arbori mai simpli, risc mai mic de overfit, dar model mai „rigid”
 - max_depth mare sau None => arbori foarte adanci, care pot invata patternuri complexe, dar cu risc de overfit
- max_features = cate trasaturi sunt luate in considerare la fiecare split
 - valori ca "sqrt", "log2" sau o fractie (ex. 0.6) introduc diversitate intre arbori si ajuta la generalizare
 - valori mari (aproape de numarul total de trasaturi) fac arborii mai asemanatori intre ei
- min_samples_leaf = numarul minim de exemple intr-o frunza
 - valori mici => frunze „inguste”, model foarte flexibil, dar mai zgomotos
 - valori mai mari => frunze mai „grase”, model mai neted, mai robust
- min_samples_split = numarul minim de exemple intr-un nod pentru a fi impartit
 - valori mici => arbori care se pot fragmenta foarte mult
 - valori mai mari => limiteaza complexitatea arborilor
- bootstrap = daca folosim sau nu esantionare cu inlocuire la antrenarea fiecarui arbore
 - True => fiecare arbore vede un bootstrap sample din date (clasicul Random Forest)
 - False => fiecare arbore se antreneaza pe tot setul, cu mai putina diversitate intre arbori

Cea mai buna configuratie gasita pentru RandomForest foloseste **200 de arbori (n_estimators = 200)**, o adancime maxima de 16 (**max_depth = 16**), **aproximativ 60% din trasaturi luate in calcul la fiecare split (max_features = 0.6)** si **bootstrap activat (bootstrap = True)**. Aceasta configuratie permite modelului sa fie suficient de expresiv pentru a surprinde relatii neliniare complexe, dar pastreaza in acelasi timp un nivel bun de robustete.

Pe setul de validare, RandomForest obtine valori de aproximativ:

- Valid_MSE ≈ 6332.68
- Valid_RMSE ≈ 79.58
- Valid_MAE ≈ 52.23
- Valid_R² ≈ 0.87

Aceste rezultate sunt mult mai bune decat cele obtinute cu Linear Regression si SVR, ceea ce arata ca un model pe arbori reuseste sa surprinda mult mai bine structura neliniara si interactiunile dintre trasaturi.

4. Gradient Boosting Regressor (loss=squared error)

Pentru acest model am folosit GradientBoostingRegressor cu funcția de pierdere standard, pătratică (loss="squared_error"). Ideea de bază este că modelul construiește arbori de decizie secvențial, fiecare arbore nou încercând să corecteze erorile făcute de toți arborii anteriori.

Hiperparametrii căutați:

- n_estimators – numărul de arbori (iterații de boosting)
 - valori mai mici $\Rightarrow$ model mai rapid, dar mai puțin expresiv
 - valori mai mari $\Rightarrow$ model mai puternic, dar cu risc de overfit și timp mai mare de antrenare
- learning_rate – "pasul" cu care fiecare arbore nou contribuie la modelul final
 - valori mici (ex. 0.03, 0.05) $\Rightarrow$ fiecare arbore contează puțin, avem nevoie de mai mulți arbori, dar modelul este mai stabil
 - valori mai mari (ex. 0.1) $\Rightarrow$ convergență mai rapidă, dar risc mai mare de overfit
- max_depth – adâncimea maximă a arborilor individuali
 - adâncime mică (2–4) $\Rightarrow$ arbori "slabi", dar care generalizează bine (tipic pentru boosting)
 - adâncime mare $\Rightarrow$ fiecare arbore devine foarte complex, riscând să memoreze zgomotul
- subsample – proporția din date folosită la fiecare arbore
 - subsample < 1.0 $\Rightarrow$ stochastic gradient boosting (mai multă diversitate, regularizare implicită)
 - subsample = 1.0 $\Rightarrow$ fiecare arbore se antrenează pe toate datele
- max_features – câte trăsături sunt luate în calcul la fiecare split
 - valori ca "sqrt", "log2" sau 0.5 $\Rightarrow$ folosim doar o parte din trăsături, ceea ce crește diversitatea și poate reduce overfitting
 - None $\Rightarrow$ se folosesc toate trăsăturile disponibile la fiecare split

Cea mai bună configurație găsită pentru GradientBoostingRegressor cu loss="squared_error" folosește:

- subsample = 1.0
- max_depth = 4
- max_features = None
- learning_rate = 0.05
- n_estimators = 300

Această combinație produce arbori relativ mici, dar suficient de adânci pentru a prinde relații neliniare, iar learning_rate = 0.07 oferă un compromis bun între stabilitate și putere de expresie.

Pe setul de validare, modelul obține aproximativ:

- Valid_MSE $\approx$ 5 675
- Valid_RMSE $\approx$ 75.33
- Valid_MAE $\approx$ 52.29
- Valid_R² $\approx$ 0.88

Comparativ cu Random Forest, Gradient Boosting cu loss="squared_error" obține un RMSE ușor mai bun pe validare, dar este depășit de RandomForest pe setul de evaluare finală. Totuși, rămâne unul dintre cele mai performante modele, dovedind că boosting-ul de arbori se potrivește foarte bine structurii neliniare a datelor.

5. Gradient Boosting Regressor (loss=quantile)

Pentru acest model am folosit tot GradientBoostingRegressor, dar cu funcția de pierdere setată pe loss="quantile", astfel încât modelul să nu mai învețe media condițională a targetului, ci anumite cuantile (de exemplu, Q10, Q50, Q90). Ideea este să putem aproxima nu doar "o singură valoare" pentru numărul de biciclete închiriate, ci și un interval de incertitudine.

Am antrenat trei modele diferite, pentru trei valori ale hiperparametrului alpha:

- alpha = 0.10 – model pentru cuantila 10% (limita inferioară);
- alpha = 0.50 – model pentru cuantila 50% (mediana);
- alpha = 0.90 – model pentru cuantila 90% (limita superioară).

Hiperparametri căutați

Pe lângă alpha, grid-ul de căutare este foarte asemănător cu cazul loss="squared_error":

- n_estimators – numărul de arbori de boosting

- `learning_rate` – contribuția fiecărui arbore la modelul final
- `max_depth` – adâncimea maximă a fiecărui arbore
- `subsample` – proporția de date folosită la fiecare arbore
- `max_features` – câte trăsături sunt luate în calcul la fiecare split

Hiperparametrul nou, specific regresiei de cuantile, este:

- `alpha` – cuantila pe care o estimează modelul:
 - `alpha = 0.5` ⇒ modelul învață mediana distribuției condiționale a lui total;
 - `alpha = 0.1` ⇒ modelul învață nivelul sub care se află $\approx 10\%$ din valori (valoare "pesimistă");
 - `alpha = 0.9` ⇒ nivelul sub care se află $\approx 90\%$ din valori (valoare "optimistă").

Cea mai bună configurație și interpretare

Pentru cuantila mediană (`alpha = 0.5`), cea mai bună configurație de hiperparametri este foarte similară cu cea de la varianta `loss="squared_error"` (arbori de adâncime mică/medie, `learning_rate` moderat, număr suficient de mare de estimatori). Acest model de mediană obține pe setul de validare:

- `Valid_MSE` $\approx 5\,508$
- `Valid_RMSE` ≈ 74.2
- `Valid_MAE` ≈ 47.3

Adică una dintre cele mai bune performanțe dintre toate modelele testate.

Modelele cu `alpha = 0.10` și `alpha = 0.90` nu sunt evaluate în principal prin MSE (scopul lor nu este să aproximeze media), ci prin modul în care împreună formează un interval de predicție. Împreună cu modelele `QuantileRegressor` ($\tau = 0.1, 0.5, 0.9$), aceste cuantile au fost folosite pentru a construi banda `[Q10, Q90]` și pentru a măsura `coverage`-ul: procentul de exemple reale pentru care total se află între predicțiile `Q10` și `Q90`. Pe setul de evaluare, intervalul `[Q10, Q90]` reușește să acopere în jur de 80% din valori, oferind astfel o estimare rezonabilă a incertitudinii.

În concluzie, `GradientBoostingRegressor` cu `loss="quantile"` are două roluri importante:

- modelul cu `alpha = 0.5` oferă o predicție robustă de tip mediană, cu performanță comparabilă sau chiar mai bună decât varianta pe MSE;
- modelele cu `alpha = 0.1` și `alpha = 0.9` permit definirea unui interval de încredere pentru numărul de biciclete închiriate, util atunci când vrem să evaluăm riscuri sau scenarii pesimiste/optimiste, nu doar o singură valoare punctuală.

6. Quantile Regressor

Pentru acest model am folosit `QuantileRegressor` pentru a estima direct cuantilele distribuției lui total, nu media. Ca și la `Gradient Boosting` cu `loss="quantile"`, scopul principal nu este să obținem cel mai mic MSE, ci să construim intervale de predicție (de exemplu `[Q10, Q90]`) care surprind incertitudinea. Au fost antrenate trei modele separate, pentru:

- `quantile = 0.1` → cuantila 10% (scenariu pesimist, limită inferioară);
- `quantile = 0.5` → cuantila 50% (mediana);
- `quantile = 0.9` → cuantila 90% (scenariu optimist, limită superioară).

Hiperparametrii folosiți

- `quantile` – cuantila pe care o aproximăm:
 - `0.1` $\approx 10\%$ din valori sunt sub acest nivel;
 - `0.5` = mediana;
 - `0.9` $\approx 90\%$ din valori sunt sub acest nivel.
- `alpha` – forța regularizării L2 (penalizare pe mărimea coeficienților):
 - `alpha` mare ⇒ coeficienți micșorați, model mai simplu, mai stabil, dar poate subestima relațiile;
 - `alpha` mic (ex. 0 sau foarte aproape de 0) ⇒ model mai flexibil, dar cu risc mai mare de supraînvățare sau instabilitate numerică.

În urma căutării de hiperparametri:

- pentru `quantile = 0.1` și `0.9` a rezultat optim `alpha = 0.0` (fără regularizare suplimentară),
- pentru `quantile = 0.5` (mediană) a rezultat optim `alpha = 0.0001`, adică o regularizare foarte slabă.
- `fit_intercept = True` – modelul învață și un termen constant (intercept), nu doar coeficienții pentru trăsături.
- `solver = "highs"` – metoda numerică folosită pentru a rezolva problema de optimizare (importantă pentru viteză și stabilitate, dar nu influențează direct interpretarea modelului).

Performanță și rol în ansamblu Pe setul de validare, `QuantileRegressor` nu excelează ca model de predicție punctuală (de exemplu, pentru cuantila de `0.5`, MSE-ul este mult mai mare decât la modelele pe arbori). Totuși, nu acesta este obiectivul principal. Punctul său forte este că:

- oferă o variantă liniară, interpretabilă, pentru estimarea cuantilelor;
- împreună cu Gradient Boosting pe cuantile, permite construirea unei benzi de incertitudine [Q10, Q90] și evaluarea coverage-ului (procentul de exemple reale care cad în acest interval).

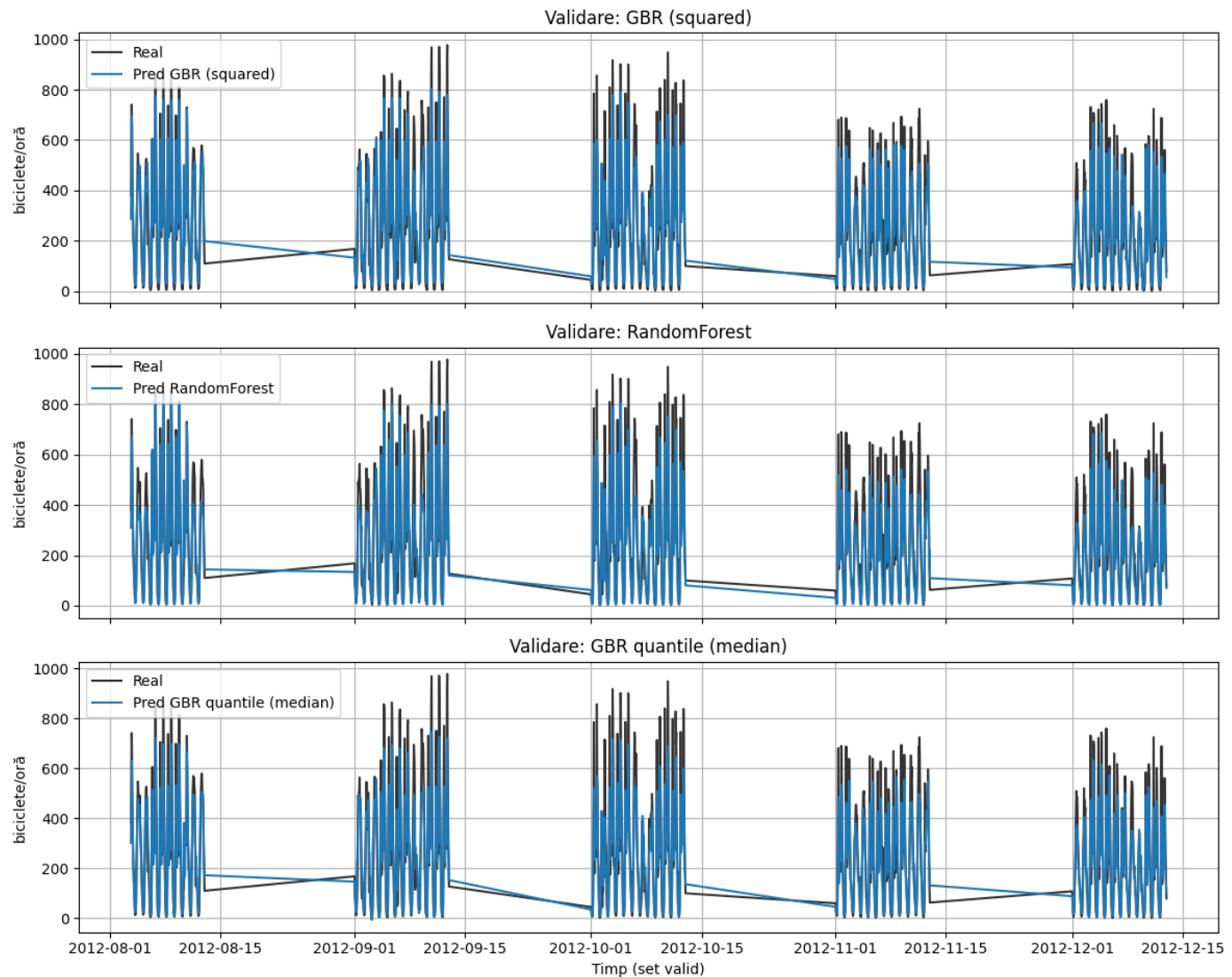
În experiment, banda [Q10, Q90] obținută din modelele de cuantile reușește să acopere aproximativ 80% din valorile reale ale lui total pe setul de evaluare, ceea ce înseamnă că intervalul este rezonabil de bine calibrat. În concluzie, QuantileRegressor nu este ales ca model final pentru predicția punctuală (acolo RandomForest și Gradient Boosting performează mult mai bine), dar este foarte util ca instrument complementar pentru estimarea intervalelor de încredere și analiza riscului (scenarii pesimiste/optimiste).

Față de GradientBoostingRegressor cu loss="quantile", QuantileRegressor este un model liniar și ușor de interpretat, care estimează cuantilele ca o combinație liniară de trăsături, dar are putere de expresie mai redusă și depinde mult de cât de bine sunt construite feature-urile. În schimb, Gradient Boosting pe cuantile este un model neliniar, bazat pe arbori, care poate surprinde relații complexe și interacțiuni între variabile, oferind de obicei performanță și intervale de predicție mai precise, în schimbul unui cost de calcul mai mare și al unei interpretabilități mai scăzute.

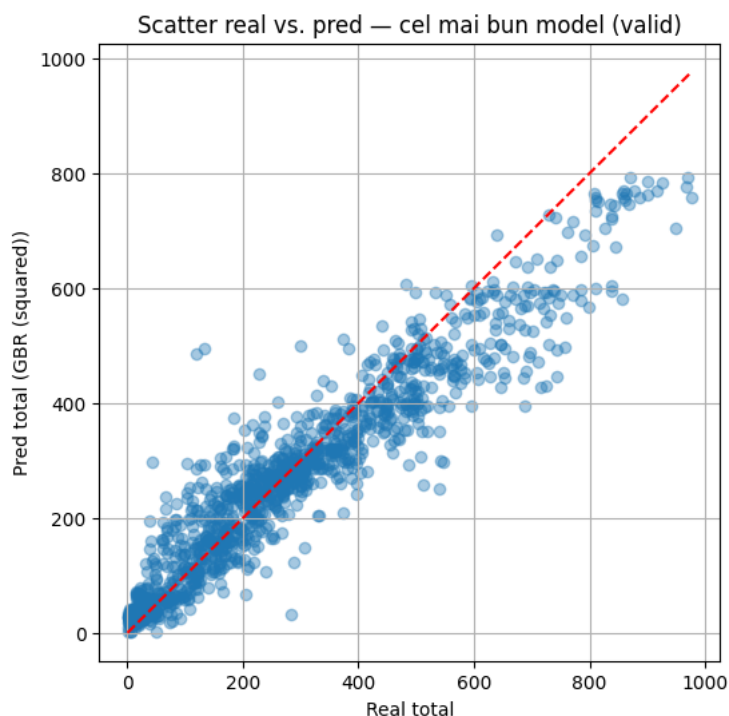
Concluzii pentru datele de antrenare

model	CV_MSE	Valid_MSE	Valid_MAE	Valid_RMSE	Valid_R2
GBR (squared)	6020.153445	5161.189391	49.910280	71.841418	0.890979
RandomForest	8488.454125	6332.684719	52.230212	79.578167	0.866233
GBR quantile (median)	8143.966240	6438.227453	50.584728	80.238566	0.864004
SVR	19265.686379	25881.653452	96.397736	160.877759	0.453296
LinearRegression (no intercept)	18672.254356	27927.801172	128.619046	167.116131	0.410075
QuantileRegressor (median)	20420.694366	32925.261040	119.199612	181.453192	0.304513

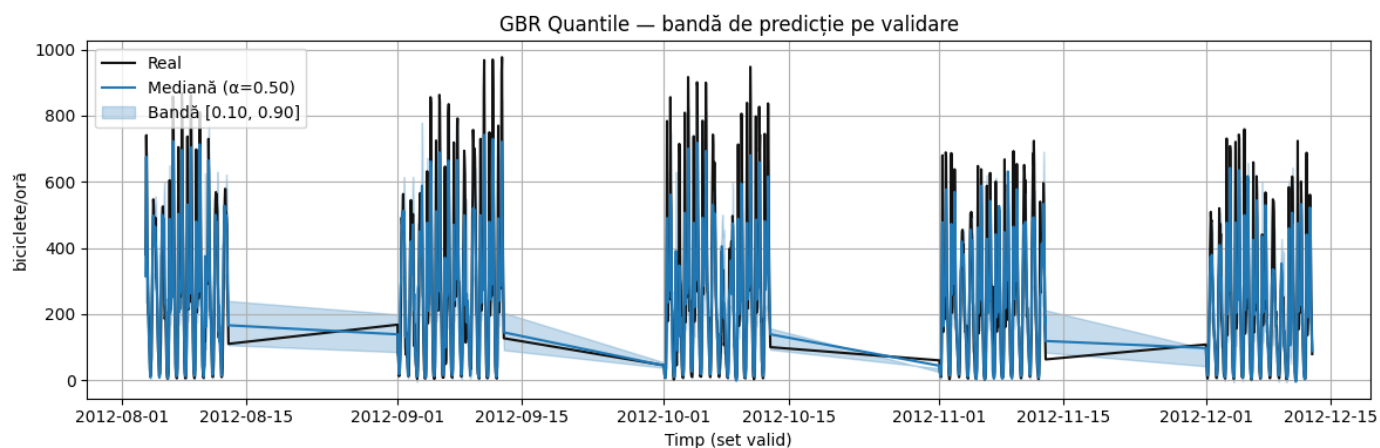
Per ansamblu, tabelul și graficele confirmă clar că modelele pe arbori sunt de departe cele mai potrivite pentru această problemă, iar modelele liniare / SVR rămân doar ca baseline-uri de comparație.



- Gradient Boosting Regressor (loss = squared_error) este cel mai bun model pe setul de validare, cu cel mai mic Valid_MSE ≈ 5161 , Valid_RMSE ≈ 71.8 și cel mai mare Valid_ $R^2 \approx 0.89$. Atât în graficele pe timp, cât și în scatter-ul real vs. pred, se vede că urmărește foarte bine forma seriilor, inclusiv vârfurile de cerere, cu abateri relativ mici.
- În scatter-ul real vs. pred se observă că majoritatea punctelor sunt aliniate strâns în jurul diagonalei, ceea ce indică o calibrare bună a modelului pe intervalele uzuale de valori. Totuși, pentru valorile foarte mari ale lui total (vârfuri de peste ~ 700 – 800 biciclete/oră), modelul tinde să subestimeze ușor cererea, punctele reale fiind deasupra diagonalei. Acest lucru sugerează că zilele sau orele "extreme", cu trafic excepțional, rămân mai greu de prezis și contribuie în principal la erorile de tip outlier.



- RandomForest este foarte aproape ca performanță (Valid_RMSE ≈ 79.6 , $R^2 \approx 0.87$), confirmând că arborii de decizie combinați în ansamblu surprind foarte bine nelinearitățile și sezonabilitatea, chiar dacă boosting-ul reușește să mai "stoarcă" câteva puncte procentuale de performanță.



- GBR quantile (median) are scoruri similare cu RandomForest (Valid_RMSE ≈ 80.2 , $R^2 \approx 0.86$), ceea ce îl face o alternativă competitivă atunci când vrem și modele de cuantile. În plus, împreună cu modelele de Q10/Q90, permite construirea benzii de predicție [0.10, 0.90] (vizibilă în ultimul grafic), care acoperă majoritatea valorilor reale și oferă informație utilă despre incertitudine.
- SVR și LinearRegression au erori de peste dublu față de modelele pe arbori (Valid_RMSE ≈ 160 – 167 , $R^2 \approx 0.41$ – 0.45), ceea ce arată că relația dintre trăsături și total este puternic nelineară și nu poate fi captată adecvat nici de un model liniar clasic, nici de SVR în configurațiile testate.
- QuantileRegressor (median) are cea mai slabă performanță punctuală (Valid_RMSE ≈ 181 , $R^2 \approx 0.30$), dar rolul lui este mai degrabă de model liniar, interpretabil de cuantile, nu de campion la MSE. El completează partea de analiză a incertitudinii, nu partea de predicție optimă.

În concluzie, pentru predicții punctuale ale numărului de biciclete închiriate, Gradient Boosting cu loss pătratic este modelul de referință pe validare, foarte apropiat de RandomForest, iar pentru analiza incertitudinii cele mai utile sunt modelele de Gradient Boosting pe cuantile, eventual completate de QuantileRegressor pentru o variantă liniară și ușor de interpretat.

Partea 4: Evaluarea finală

După alegerea hiper-parametrilor pe setul de antrenare/validare, am refăcut antrenarea fiecărui model folosind toate datele disponibile pentru train (train + valid) și am evaluat performanța pe un set separat, eval_split.csv, care joacă rol de test final. Preprocesarea (pipeline-ul de imputare, standardizare, encodare și selecție de atribute) a

rămas aceeași ca în etapele anterioare. Pentru evaluare am folosit aceleași metrice: MSE, MAE, RMSE și R².

Tabelul de mai jos sintetizează rezultatele:

- **RandomForest** este modelul cu cele mai bune scoruri pe setul eval_split:
 - Eval_MSE ≈ 2312.9, Eval_RMSE ≈ 48.1, Eval_MAE ≈ 29.0, Eval_R² ≈ 0.93. Aceste valori indică o generalizare foarte bună și o potrivire strânsă între valorile reale și cele prezise. Față de celelalte modele, RandomForest reușește să mențină cea mai mică eroare și cel mai mare R², motiv pentru care este ales ca model final pentru predicții punctuale.
- **GradientBoostingRegressor** (loss = quantile, mediană) și GBR (loss = squared_error) vin imediat în urma lui RandomForest, cu performanțe foarte apropiate:
 - GBR quantile (median): Eval_RMSE ≈ 52.7, Eval_R² ≈ 0.92;
 - GBR (squared): Eval_RMSE ≈ 54.2, Eval_R² ≈ 0.91. Diferențele de ordinul câtorva biciclete/oră arată că toate cele trei modele pe arbori generalizează bine. Faptul că pe validare cel mai bun era GBR (squared), iar pe eval RandomForest trece ușor înainte sugerează că nu avem overfitting semnificativ pe setul de validare, ci doar mici variații datorate perioadelor de timp diferite.
- **Modelele liniare** și **SVR** rămân semnificativ în urmă:
 - SVR: Eval_RMSE ≈ 127.2, Eval_R² ≈ 0.52;
 - LinearRegression (no intercept): Eval_RMSE ≈ 133.8, Eval_R² ≈ 0.47;
 - QuantileRegressor (median): Eval_RMSE ≈ 138.6, Eval_R² ≈ 0.43. Aceste valori confirmă observațiile din etapa de validare: relația dintre trăsături și target este puternic neliniară, iar modelele liniare (și SVR, în configurațiile testate) nu reușesc să o surprindă suficient de bine.

Pentru modelele de cuantile, am evaluat și acoperirea benzii [0.10, 0.90] pe setul eval_split:

- banda construită cu GBR quantile (Q10 și Q90) acoperă aproximativ 73% din valori;
- banda construită cu QuantileRegressor (Q10–Q90) acoperă aproximativ 80.1% din valori.

În practică, aceasta înseamnă că, pe lângă predicția punctuală dată de RandomForest, putem oferi și un interval de încredere pentru numărul de biciclete închiriate, derivat din modelele de cuantile, util pentru scenarii de tip „pesimist/optimist”.

model	Eval_MSE	Eval_MAE	Eval_RMSE	Eval_R2
RandomForest	2312.866740	28.971408	48.092273	0.931354
GBR (squared)	2673.763860	33.779231	51.708451	0.920642
GBR quantile (median)	2945.957169	33.510394	54.276672	0.912563
SVR	14449.208671	69.625727	120.204861	0.571145
LinearRegression (no intercept)	17903.671875	97.192912	133.804603	0.468616
QuantileRegressor (median)	19240.823541	89.220535	138.711296	0.428929

În concluzie, evaluarea finală pe eval_split confirmă că:

- **RandomForest** este alegerea naturală pentru modelul final de predicție punctuală (cel mai bun RMSE și R² pe setul de test);
- **Gradient Boosting** pe cuantile și QuantileRegressor completează soluția, oferind benzi de predicție care descriu incertitudinea estimărilor și pot fi folosite la decizii de capacitate sau planificare.

Concluzie

În concluzie, analiza exploratorie, preprocesarea și experimentele cu mai mulți algoritmi au arătat că modelele pe arbori (RandomForest și Gradient Boosting) sunt mult mai potrivite pentru problema de față decât modelele liniare sau SVR, reușind să surprindă atât nelinearitățile, cât și sezonabilitatea din date. Pipeline-ul de preprocesare (extragere de trăsături temporale, encodare, standardizare, selecție de atribute) s-a dovedit esențial pentru obținerea acestor rezultate, iar căutarea de hiper-parametri a evidențiat impactul direct al acestora asupra performanței fiecărui model. Evaluarea finală pe setul eval_split confirmă RandomForest ca model recomandat pentru predicții punctuale, în timp ce modelele de cuantile bazate pe Gradient Boosting și QuantileRegressor oferă benzi de predicție utile pentru a cuantifica incertitudinea.

AUTOVIT

Acest raport detaliază procesul de analiză, preprocesare și antrenare a unor modele de învățare automată pentru un set de date cu anunțuri auto (Autovit). Obiectivul final este predicția prețului unui autoturism (respectiv a lui pret / log_pret), folosind informațiile disponibile în anunț: caracteristici tehnice (an fabricație, rulaj, motorizare, putere, normă de poluare, transmisie), detalii comerciale (preț, negociabil, stare) și alte atribute descriptive. Scopul raportului este de a identifica o combinație de pași de preprocesare și modele care să ofere o estimare cât mai precisă a prețului, dar și de a înțelege care factori influențează cel mai mult valoarea unui autoturism.

Partea 1: Analiza Exploratorie a Datelor (EDA)

În această secțiune analizăm structura setului de date Autovit, calitatea datelor (valori lipsă) și principalii factori care influențează prețul unei mașini.

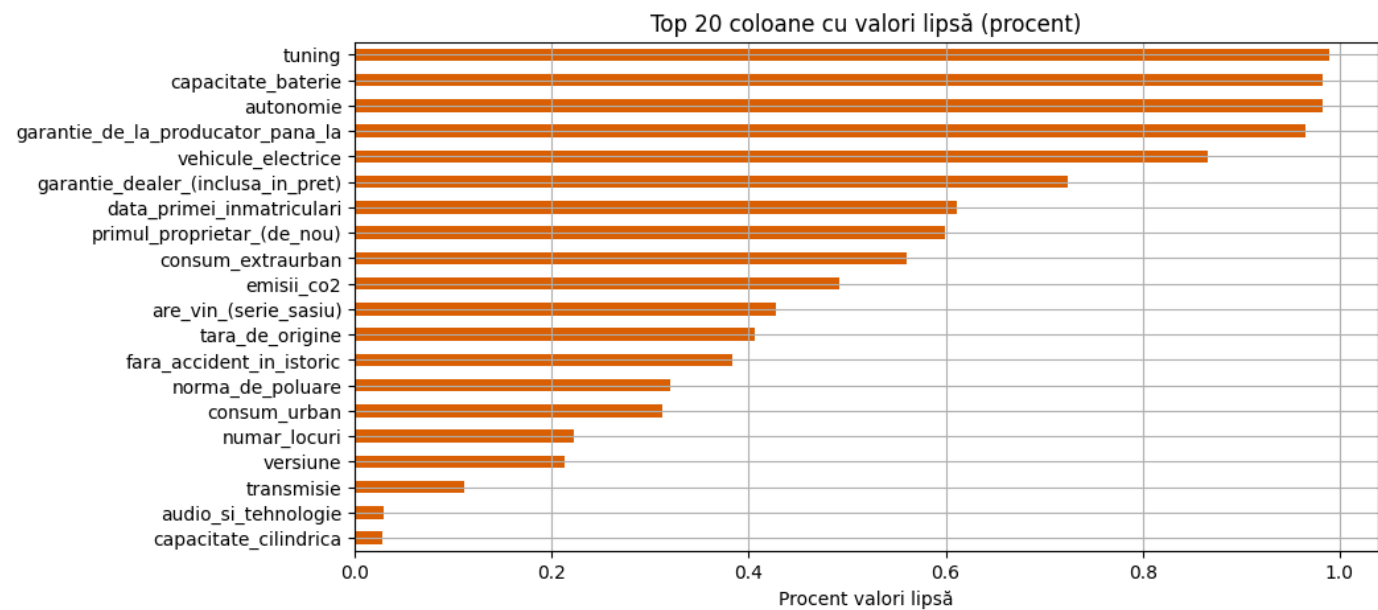
1. Structura setului si valori lipsa
- Setul de date conține 18 988 de înregistrări și 39 de coloane, dintre care:

- 12 coloane numerice (ex. pret, km, putere, capacitate_cilindrica, log_km, log_pret),
- 26 coloane de tip object (categorice sau text),
- 1 coloană de tip dată (data_primei_inmatriculari).

Un exemplu de rând arată că pentru fiecare anunț avem informații despre marca, modelul, anul fabricației, rulaj, combustibil, putere, cilindree, transmisie, tip caroserie, culoare, stare și diverse liste de dotări (audio, confort, siguranță etc.).

id	nume	pret	oferit_de	are_vin_(serie_sasiu)	marca	model	versiune	anul_fabricației	km	combustibil	putere	ca
0	Opel Astra	3150.0	Firma	NaN	Opel	Astra	NaN	2002	160000.0	Benzina	100.0	159
1	Renault Megane	12495.0	Firma	NaN	Renault	Megane	NaN	2017	159242.0	Diesel	110.0	146
2	BMW X4 xDrive20d Aut. M Sport	24990.0	Firma	Da	BMW	X4	xDrive20d Aut. M Sport	2016	145000.0	Diesel	190.0	196
3	Volkswagen Passat 2.0 TDI DSG Highline	22850.0	Firma	Da	Volkswagen	Passat	2.0 TDI DSG Highline	2019	77874.0	Diesel	150.0	196
4	Volkswagen Golf	7199.0	Firma	Da	Volkswagen	Golf	NaN	2010	230000.0	Benzina	122.0	136

Pentru a verifica calitatea datelor am calculat proporția de valori lipsă pe fiecare coloană și am reprezentat top 20 coloane cu cele mai multe valori lipsă.



Se observă că:

- anumite coloane specifice vehiculelor electrice sau situațiilor particulare au peste 95% valori lipsă: tuning (98.9%), capacitate_baterie (98.3%), autonomie (98.2%), garantie_de_la_producator_pana_la (96.5%), vehicule_electrice (86.5%);
- alte coloane cu lipsuri mari (60–70%) sunt: garantie_dealer_(inclusa_in_pret), data_primei_inmatriculari, primul_proprietar_(de_nou);

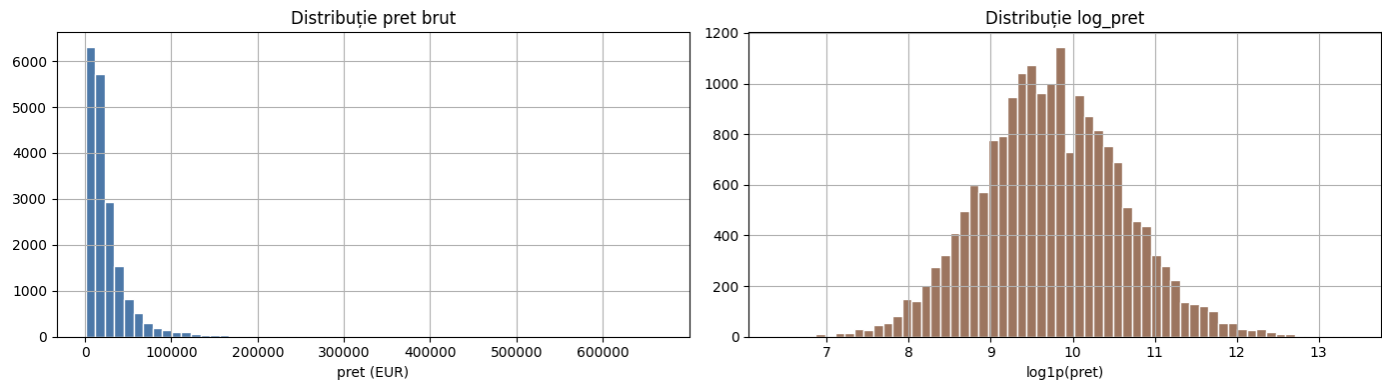
- există și coloane importante cu lipsuri moderate: consum_extraurban, emisii_co2, are_vin_(serie_sasiu), tara_de_origine, fara_accident_in_istoric, norma_de_poluare, consum_urban, numar_locuri, versiune, transmisie.

Coloanele cu peste ~80–90% valori lipsă vor fi, în general, eliminate sau folosite marginal, în timp ce pentru restul este necesar un pas de imputare în pipeline (mediana pentru numerice, cea mai frecventă valoare pentru categorice).

#	Column	Non-Null Count	Dtype
0	nume	18988	object
1	pret	18988	float64
2	oferit_de	18988	object
3	are_vin_(serie_sasiu)	10877	object
4	marca	18988	object
5	model	18988	object
6	versiune	14946	object
7	anul_fabricației	18988	int64
8	km	18945	float64
9	combustibil	18988	object
10	putere	18983	float64
11	capacitate_cilindrica	18447	float64
12	transmisie	16872	object
13	consum_extraurban	8352	float64
14	cutie_de_viteze	18987	object
15	consum_urban	13064	float64
16	tip_caroserie	18988	object
17	emisii_co2	9635	float64
18	numar_de_portiere	18759	float64
19	culoare	18988	object
20	numar_locuri	14757	float64
21	garantie_dealer_(inclusa_in_pret)	5247	object
22	primul_proprietar_(de_nou)	7620	object
23	fara_accident_in_istoric	11701	object
24	stare	18988	object
25	audio_si_tehnologie	18439	object
26	confort_si echipamente_optionale	18866	object
27	electronice_si_sisteme_de_asistenta	18678	object
28	siguranta	18474	object
29	norma_de_poluare	12914	object
30	tara_de_origine	11275	object
31	data_primei_inmatriculari	7390	datetime64[ns]
32	garantie_de_la_producator_pana_la	670	object
33	vehicule_electrice	2555	object
34	tuning	200	object
35	autonomie	338	object
36	capacitate_baterie	317	object
37	log_km	18945	float64
38	log_pret	18988	float64

2. Distributia target-ului (pret si log_pret)

Targetul problemei este pret, iar în preprocesare folosim și varianta sa logaritmică log_pret. Histogramelor de mai jos arată distribuția prețului brut și a prețului logaritmic.



Din statistici descriptive pe cuantile:

metric	pret	log_pret
median	16500.0	9.711176
p90	52900.0	10.876178
p95	72984.3	11.198013
max	666666.0	13.410046

Distribuția pret este puternic asimetrică spre dreapta: majoritatea mașinilor sunt în jurul a câteva zeci de mii de euro, dar există și anunțuri cu prețuri extrem de mari (peste 200 000–300 000 EUR), vizibile în coada distribuției și în histogramă.

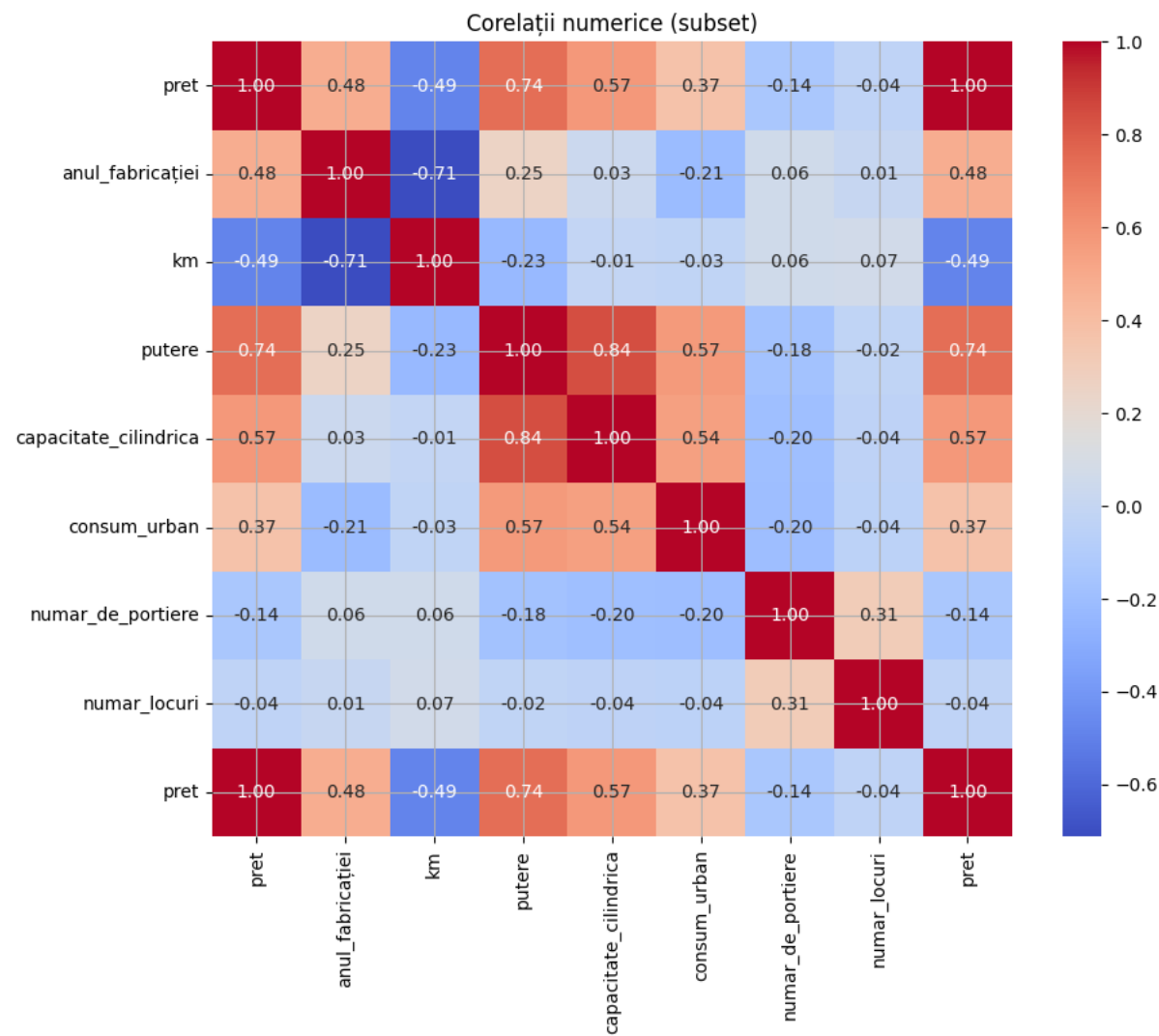
Transformând prețul în log_pret, distribuția devine mult mai simetrică și “clopot”, ceea ce ajută modelele (în special cele liniare) și stabilizează variația între observații.

Concluzie: vom lucra preponderent cu log_pret ca target în antrenare, dar vom evalua erorile și în termeni de preț real, raportând metriche precum MAE și RMSE pentru a înțelege eroarea în euro.

3. Relatia dintre pret si variabile numerice

Identificăm cei mai puternici predicatori numeric (km, vechime, putere etc.) pentru a prioritiza transformări/discretizări.

Pentru variabilele numerice relevante am calculat o matrice de corelație Pearson, reprezentată sub formă de heatmap.



Principalele corelații ale lui pret sunt:

- pozitive puternice cu:
 - putere (≈ 0.74) – mașinile cu putere mai mare sunt, în medie, mai scumpe;
 - capacitate_cilindrica (≈ 0.57) – cilindrul mai mare este asociat cu preț mai mare;
 - anul_fabricației (≈ 0.48) – mașinile mai noi sunt mai scumpe;
 - consum_urban (≈ 0.37) – indirect, modele mai puternice/grele tind să aibă și consum urban mai mare.
- negativă cu:
 - km (≈ -0.49) – rulajul mare reduce prețul.

Variabile precum numar_de_portiere sau numar_locuri au corelații slabe cu prețul, ceea ce sugerează că ele au un impact secundar.

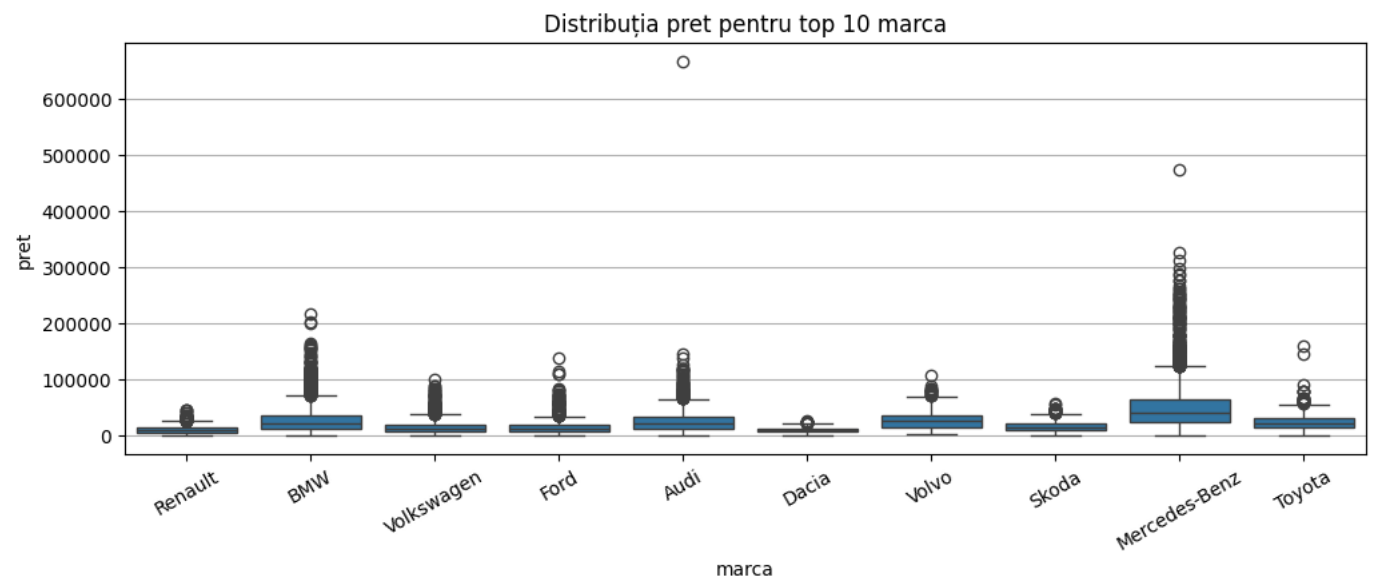
Cele mai predictive variabile numerice pentru preț sunt putere, capacitate_cilindrica, anul_fabricației, km și, indirect, consum_urban. Aceste relații sunt însă neliniare (de ex. scăderea prețului cu kilometrii nu e perfect liniară), motiv pentru care modelele pe arbori vor fi probabil mai potrivite decât o regresie liniară simplă.

4. Impactul categoriilor (marca, combustibil, transmisie)

Prețul variază semnificativ în funcție de marca și tipul mașinii; graficele boxplot ajută la justificarea encodării categorice și eventual la gruparea valorilor rare.

Marca

Boxplotul pentru top 10 mărci arată diferențe clare între segmente.

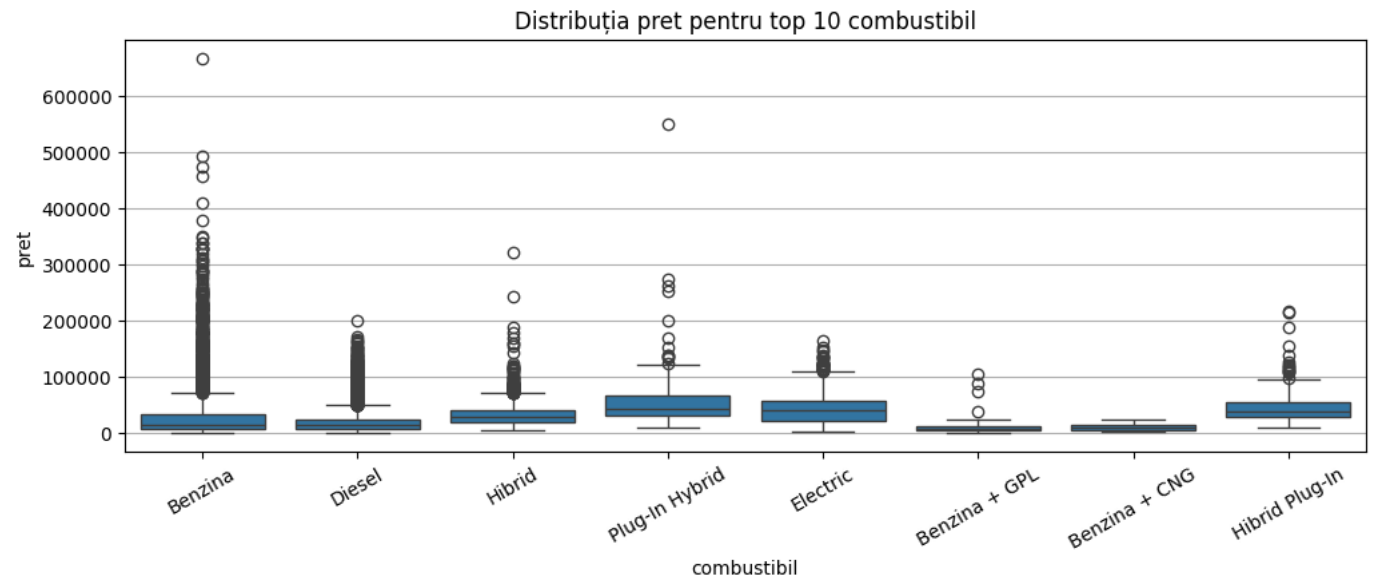


- Mărci de volum precum Renault, Volkswagen, Ford, Dacia au mediane de preț mai joase.
- Mărci premium (BMW, Audi, Mercedes-Benz, Volvo) au mediane semnificativ mai mari și o variație internă mai mare (cozi lungi spre prețuri ridicate).
- Pentru anumite mărci (ex. Mercedes-Benz, Audi) apar outlieri foarte mari, corespunzând modelelor de lux sau foarte noi.

Combustibil

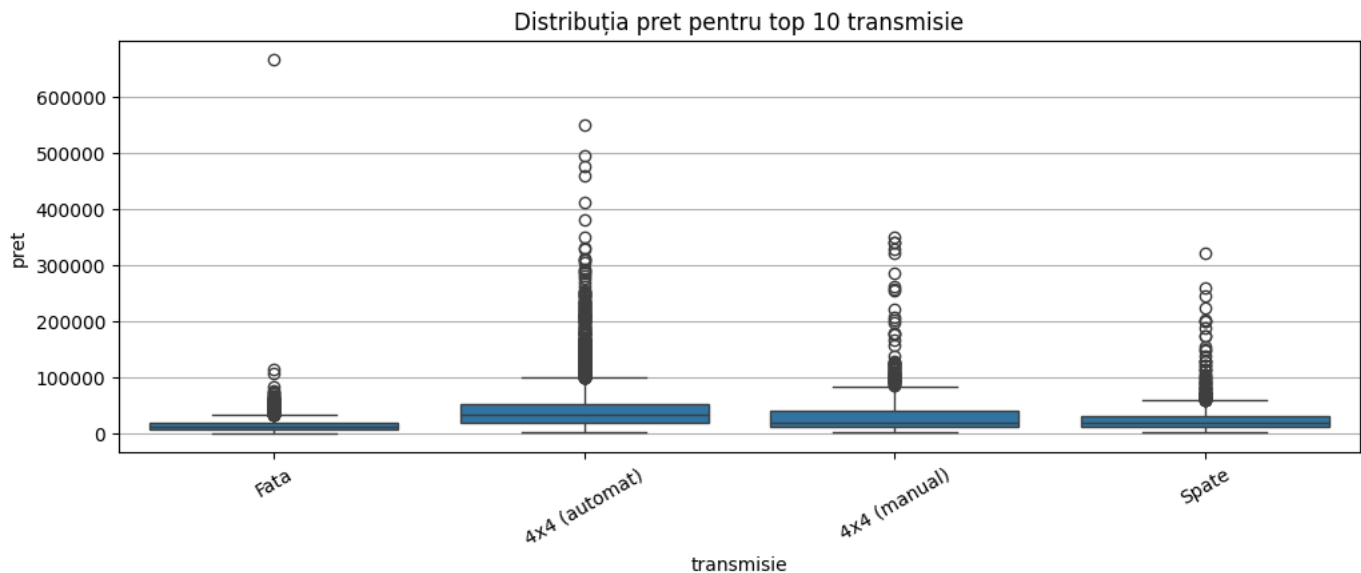
Boxplotul pentru top 10 tipuri de combustibil evidențiază că:

- Mașinile hibride, plug-in hybrid și electrice tind să aibă mediane mai mari decât cele pe benzină sau diesel, cu o dispersie ridicată a prețurilor.
- Tipuri alternative (GPL, CNG) sunt concentrate în general în segmente mai ieftine.
- Cozile lungi arată că există atât modele de intrare, cât și modele foarte scumpe pentru aceleași tipuri de combustibil.



Transmisie

Distribuția prețului pentru principalele tipuri de transmisie este prezentată în următorul boxplot.



- Mașinile cu tracțiune față au, în medie, cele mai mici prețuri.
- Configurațiile 4x4 (automat/manual) și tracțiunea spate au mediane mai ridicate și multe outlieri spre prețuri foarte mari, ceea ce indică modele mai scumpe (SUV-uri, mașini sport, premium).

Per total, EDA pe setul Autovit arată că:

- datele sunt relativ bogate, dar conțin mai multe coloane cu valori lipsă foarte multe și atribute text complexe;
- pret are o distribuție foarte asimetrică, iar utilizarea lui `log_pret` este esențială pentru o modelare stabilă;
- atât variabilele numerice (km, vechime/anul_fabricației, putere, capacitate_cilindrică), cât și cele categorice (marca, combustibil, transmisie) influențează puternic prețul și vor sta la baza pipeline-ului de preprocesare și a modelelor prezentate în secțiunile următoare.

Partea 2: Preprocesare (imputare, standardizare, encodare, discretizare, selecție a atributelor)

Pentru aceasta parte, vom începe cu stabilirea atributelor relevante, care vor intra în pipeline-uri. Coloanele de tip text liber sau listele de dotari (cum sunt „audio_si_tehnologie”, „confort_si_echipamente_optionale”, „electronice_si_sisteme_de_asistenta”, „siguranta”, dar și coloane aproape complet goale precum „tuning”, „capacitate_baterie”, „autonomie”, „garantie_de_la_producator_pana_la”, „vehicule_electrice”) ar necesita un tratament special separat, dar, combinat cu faptul că prezintă aproximativ 80-90% din valori lipsă, vom alege să le eliminăm din pipeline-urile principale. Pentru aceasta parte, ne vom concentra pe variabilele clasice, numerice și categorice, care descriu structura mașinii: marca, modelul, anul fabricației, kilometrii, combustibilul, transmisia, capacitatea cilindrică, norma de poluare, starea, țara de origine etc. Targetul pentru acest dataset este prețul, dar pentru analiză vom folosi `log_pret`, fiind mult mai utilă, introducând o distribuție normală, care ne va ajuta mult mai mult în analizele noastre.

Setul de date a fost împărțit ulterior în 2 părți, 80% pentru partea de train, iar 20% pentru partea de testare, iar atributele au fost împărțite în cele două tipuri, coloane numerice: anul_fabricației, km, putere, capacitate_cilindrică, consum_urban, consum_extraurban, emisii_co2, numar_de_portiere, numar_locuri, log_km; și coloane categorice: oferit_de, marca, model, combustibil, transmisie, cutie_de_viteze, tip_caroserie, culoare, stare, norma_de_poluare, tara_de_origine, etc.

Pentru variabilele numerice am folosit un pipeline simplu, dar robust: mai întâi imputare cu mediana, apoi standardizare. Imputarea cu mediana rezolvă problema valorilor lipsă apărute în câmpuri precum km, putere sau emisii_co2 și este rezistentă la outlieri (spre deosebire de media, care ar fi trasă în sus de câteva valori extreme). Standardizarea aduce toate aceste variabile pe o scară comparabilă, cu medie aproximativ 0 și deviație standard 1, lucru important în special pentru modele sensibile la scară, cum ar fi SVR sau regresia liniară regularizată, dar util și în general pentru stabilitatea optimizării.

```
numeric_pipeline = Pipeline([("imp", SimpleImputer(strategy="median")), ("scaler", StandardScaler())])
```

Pentru variabilele categorice, preprocesarea a fost ceva mai complexă. În primul rând, pentru a nu crea mii de coloane după One Hot Encoding, am folosit un `RareLabelEncoder`, care grupează toate valorile foarte rare ale unei variabile într-o singură categorie „Other” (am folosit un prag de frecvență de aproximativ 1%). Astfel, mărcile sau versiunile care apar în doar câteva zeci de anunțuri sunt agregate și nu mai produc dummies cu foarte puține observații. Apoi, valorile lipsă din categorice (de exemplu la transmisie sau norma_de_poluare) sunt completate cu cea mai frecventă categorie, iar la final se aplică un `OneHotEncoder` cu `handle_unknown="ignore"`, astfel încât pipeline-ul să poată gestiona și categorii noi apărute în setul de validare sau în date viitoare.

```
categorical_pipeline = Pipeline([("rare", rare_encoder), ("imp", SimpleImputer(strategy="most_frequent")), ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
```

Pe lângă reprezentarea continuă a unor variabile, am introdus și o discretizare în binuri pentru câteva coloane cheie, în special pentru km și vechime. Aceste variabile au fost trecute printr-un `KBinsDiscretizer` cu 5 binuri pe cuantile, urmat de o encodare one-hot. În practică asta înseamnă că, pe lângă informația „kilometri = 150 000”, modelul primește și informația „mașină cu rulaj ridicat” (prin binul corespunzător), ceea ce îl ajută să învețe mai ușor efecte de tip prag – de exemplu scăderea bruscă a prețului după un anumit rulaj sau după o anumită vechime. În `ColumnTransformer`, km și vechime sunt prezente atât ca variabile numerice standardizate, cât și ca binuri one-hot, oferind modelelor două vederi complementare asupra aceluiași fenomen.

```
bin_pipeline = Pipeline([("imp", SimpleImputer(strategy="median")), ("kbin", KBinsDiscretizer(n_bins=5, encode="onehot-dense", strategy="quantile"))])
```

Toate aceste bucăți au fost combinate într-un singur `ColumnTransformer` numit `preprocess`, care aplică automat pipeline-ul numeric pe toate coloanele numerice, pipeline-ul categoric pe toate coloanele categorice și pipeline-ul de binuri pe variabilele selectate pentru discretizare. Rezultatul este o matrice numerică densă,

X_train_proc / X_valid_proc, ce conține variabile numerice scalate, categorice OHE și binuri pentru km și vechime. Aceasta este baza unică pe care sunt antrenate ulterior toate modelele, astfel încât comparația între algoritmi să fie corectă.

```
preprocess = ColumnTransformer( transformers=[ ("num", numeric_pipeline, NUMERIC_STD), ("cat", categorical_pipeline, CAT_COLS), ("bins", bin_pipeline, BIN_COLS) ], remainder="drop", sparse_threshold=0.0 )
```

Înainte de preprocesare, numărul de atribute era de 31, iar după aplicarea tuturor transformărilor, am ajuns la 191.

```
Split sizes: (15190, 31) (3798, 31) Preprocessed shapes: (15190, 191) (3798, 191)
```

Pentru a nu lucra cu un număr prea mare de trăsături după encodare și pentru a scoate în evidență atributele cu adevărat relevante, am inclus și un pas de selecție a atributelor. Am testat două familii de metode: SelectKBest, bazat pe informație mutuală între fiecare trăsătură și target, și SelectFromModel cu un model Lasso, care introduce regularizare L1 și împinge coeficienții neimportanți la zero. Pentru SelectKBest am încercat mai multe valori ale lui k (de exemplu 30, 60, 90, 95, 120 și fracții procentuale din numărul total de trăsături), iar pentru fiecare am antrenat un model Ridge de referință și am măsurat RMSE pe setul de validare. Pentru SelectFromModel am variat hiperparametrul alpha (0.0005, 0.001, 0.005, 0.01), păstrând trăsăturile cu importanță peste media coeficienților absoluți.

#	config	n_features	Valid_RMSE
0	SelectKBest k=120	120	15351.153860
1	SelectKBest k=90	90	15399.217726
2	SelectKBest k=95	95	15474.841316
3	SelectKBest k=60	60	15539.047934
4	SelectKBest k=57	57	15543.170281
5	SelectKBest k=30	30	15710.226468
6	SelectFromModel Lasso $\alpha=0.005$	56	15863.217038
7	SelectFromModel Lasso $\alpha=0.001$	60	15865.099696
8	SelectFromModel Lasso $\alpha=0.0005$	59	15865.538543
9	SelectFromModel Lasso $\alpha=0.01$	55	15874.785180

Rezultatele acestor experimente arată că cea mai bună configurație este SelectKBest cu k = 120, care obține cel mai mic RMSE pe validare, imediat urmată de variante cu 90–95 de trăsături. Metodele bazate pe Lasso reușesc să reducă și mai mult numărul de atribute, dar cu un cost mic de performanță. Concluzia este că reducerea dimensionalității de la setul brut (după toate encodările) la aproximativ 120 de trăsături păstrează aproape integral informația utilă pentru predicția prețului și simplifică modelul. În plus, faptul că SelectKBest păstrează în principal trăsături legate de km, vechime, putere, capacitate_cilindrica, precum și dummies pentru cele mai frecvente mărci, tipuri de combustibil și transmisii confirmă observațiile din EDA: acești factori sunt cei mai importanți pentru evaluarea prețului unei mașini.

Partea 3: Modele + Tuning + Evaluare

Toate cele 6 modele au folosit RandomizedSearchCV pentru căutarea hiper-parametrilor și 5-fold Cross-Validation (KFold cu amestecarea datelor) pentru estimarea performanței pe train. Toate metodele folosesc același pipeline: preprocesare → selecție de atribute → model, iar căutarea de hiper-parametri testează diverse combinații pentru fiecare algoritm, cu scopul de a găsi configurația care minimizează MSE.

Explicații despre cum funcționează fiecare algoritm în general:

Linear Regression Regresia liniară pornește de la ideea că targetul poate fi aproximat ca o combinație liniară de trăsături. Modelul caută coeficienții w care minimizează suma pătratelor erorilor dintre valorile reale și cele prezise (MSE). Intuitiv, încearcă să „tragă” un hiper-plan într-un spațiu de dimensiune mare care să treacă cât mai aproape de toate punctele. Este un model simplu, ușor de interpretat (fiecare coeficient spune cât crește/scade targetul când crește o trăsătură cu o unitate), dar are dificultăți în a surprinde relații puternic neliniare sau interacțiuni complexe între variabile.

SVR (Support Vector Regressor) SVR este versiunea pentru regresie a mașinilor cu vectori suport (SVM). Ideea de bază este să găsească o funcție care să fie „cât mai plată” (simplă), dar să se afle în interiorul unui tub de grosime ϵ în jurul datelor, ignorând erorile mici și penalizând doar punctele care ies în afara acestui tub. Cu ajutorul kernel-urilor (de exemplu RBF), SVR proiectează datele într-un spațiu de dimensiune mai mare, în care relații neliniare devin aproximativ liniare, și în acel spațiu caută această funcție plată. Modelul se concentrează pe un subset de puncte „suport” (cele care forțează marginile tubului), ceea ce îi dă un control fin asupra compromisului dintre complexitate și eroare.

Random Forest Regressor Random Forest este un ansamblu de arbori de decizie. În loc să antrenăm un singur arbore profund (care ar overfita ușor), antrenăm mulți arbori, fiecare pe un eșantion ușor diferit al setului de date (bootstrap) și folosind doar o submulțime aleatoare de trăsături la fiecare split. Fiecare arbore produce o predicție, iar rezultatul final este media acestor predicții. Această „în medie” peste mulți arbori reduce varianta și tinde să anuleze erorile individuale ale arborilor, păstrând în același timp capacitatea lor de a surprinde relații foarte neliniare și interacțiuni între variabile.

Gradient Boosting Regressor (loss = squared_error) Gradient Boosting construiește tot un ansamblu de arbori, dar într-un mod secvențial, nu independent. Se pornește de la o predicție simplă (de exemplu media targetului), apoi la fiecare pas se antrenează un arbore mic care să corecteze erorile modelului curent (diferența dintre valorile reale și cele prezise până atunci). Noul model este suma dintre modelul anterior și un pas mic în direcția acestui arbore corector, controlat de learning_rate. Repetând procesul de zeci sau sute de ori, modelul învață treptat un ansamblu de arbori care aproximează foarte bine funcția target. Cu loss="squared_error", criteriul de optimizare este MSE, deci modelul este ajustat pentru a minimiza eroarea pătratică medie.

Gradient Boosting Regressor (loss = quantile) Varianta cu loss="quantile" folosește același mecanism de boosting (arbori adăugați secvențial), dar schimbă funcția de pierdere astfel încât să învețe o cantilă a distribuției, nu media. Pentru un alpha dat (de exemplu 0.1, 0.5, 0.9), modelul este antrenat astfel încât aproximativ alpha din valori să fie sub predicție și restul deasupra. Rezultatul este un model care, în loc să spună „preț estimat este X”, spune „preț sub care se află 10% /

50% / 90% dintre valori este X'' . Antrenând modele separate pentru Q10, Q50 și Q90, se pot construi intervale de predicție care cuantifică incertitudinea (bandă [Q10, Q90]).

Quantile Regressor QuantileRegressor este o regresie liniară adaptată pentru cuantile. În loc să minimizeze MSE, minimizează o funcție de pierdere asimetrică (pinball loss) pentru o cantilă τ (ex. 0.1, 0.5, 0.9). Practic, penalizează diferit erorile de sub- și supra-predicție, astfel încât să alinieze modelul cu „nivelul” respectiv al distribuției. Forma modelului este tot liniară, deci coeficienții se interpretează ușor, dar, fiind liniar, nu poate surprinde relații foarte complexe. Avantajul este simplitatea: pentru fiecare cantilă ai un set de coeficienți care îți spun cum influențează trăsăturile limita inferioară/superioară a prețului.

RandomizedSearchCV RandomizedSearchCV este o metodă de căutare a hiper-parametrilor care, în loc să testeze exhaustiv toate combinațiile dintr-un grid (ca GridSearchCV), alege aleator un număr limitat de combinații din distribuții prestabilite pentru fiecare hiper-parametru. Pentru fiecare combinație aleasă, antrenează modelul pe mai multe folduri de cross-validation și calculează o metrică medie (de exemplu MSE). La final, reține setul de hiper-parametri care a obținut cel mai bun scor mediu. Avantajul este că, pentru același timp de calcul, poate explora un spațiu mult mai mare de combinații decât grid search, ceea ce e foarte util când avem mulți hiper-parametri (de exemplu la RandomForest sau Gradient Boosting) și nu știm dinainte unde se află zona „bună” din spațiul lor.

1. Linear Regression

Pentru acest algoritm am testat trei variante:

- varianta standard;
- fără intercept (forțează modelul să treacă prin origine);
- cu coeficienți doar pozitivi (toți coeficienții sunt constrânși să fie ≥ 0).

Cea mai bună configurație a fost varianta standard (cu intercept, fără constrângerea coeficienților pozitivi). Pe setul de validare, această variantă obține:

- Valid_MSE $\approx 2.37 \cdot 10^8$
- Valid_RMSE $\approx 15\,403$
- Valid_MAE $\approx 8\,095$
- Valid_R² ≈ 0.74

Deși la prima vedere R² ≈ 0.74 nu este rău, valorile RMSE și MAE sunt semnificativ mai mari decât la modelele pe arbori. Acest lucru indică faptul că relația dintre trăsături și pret este puternic neliniară, iar un model liniar simplu nu reușește să o surprindă suficient de bine. În consecință, regresia liniară rămâne un baseline rezonabil, dar este clar depășită de modelele mai complexe.

2. SVR (Support Vector Regressor)

Pentru SVR am variat aceiași hiper-parametri ca în studiul precedent: kernel (linear / RBF), C, epsilon, gamma. Căutarea a preferat, și aici, o configurație cu kernel RBF, C mare și gamma moderat, ceea ce permite modelului să surprindă relații neliniare între trăsături și preț.

Cea mai bună configurație găsită de RandomizedSearchCV este (adică un SVR liniar, cu regularizare slabă – C mare – și un gamma mic, dar acesta contează efectiv doar pentru kernel RBF):

```
kernel = "linear" C = 100 epsilon = 0.05 gamma = 0.03
```

Cea mai bună configurație găsită pentru SVR obține pe setul de validare:

- Valid_MSE $\approx 3.24 \cdot 10^8$
- Valid_RMSE $\approx 18\,008$
- Valid_MAE $\approx 6\,946$
- Valid_R² ≈ 0.65

Chiar dacă modelul este neliniar, performanța este mai slabă decât a regresiei liniare pe acest set (R² mai mic și RMSE mai mare), semn că SVR, în configurațiile testate, nu reușește să se adapteze la complexitatea și dimensiunea datelor de tip anunț auto. Comparativ cu modelele pe arbori, SVR este net inferior și rămâne doar ca metodă de comparație.

3. Random Forest Regressor

Pentru RandomForestRegressor am căutat hiper-parametri de tipul:

- n_estimators (numărul de arbori),
- max_depth (adâncimea maximă),
- max_features (numărul de trăsături testate la fiecare split),
- min_samples_leaf, min_samples_split,
- bootstrap (cu / fără sampling cu înlocuire).

Căutarea a favorizat combinații cu număr suficient de mare de arbori, adâncime moderată și un max_features sub numărul total de trăsături, ceea ce crește diversitatea arborilor și stabilizează predicțiile.

Cea mai bună combinație găsită este:

```
n_estimators = 600 max_depth = 12 max_features = 0.3 bootstrap = False
```

(min_samples_split și min_samples_leaf au valori moderate, alege automat de best_params)

Adică o pădure cu 600 de arbori, fiecare de adâncime maximă 12, care la fiecare split ia în considerare aproximativ 30% din trăsături, fără bootstrap (fiecare arbore vede tot setul, dar cu randomizare pe trăsături și splits).

Pe setul de validare, Random Forest obține:

- Valid_MSE $\approx 9.46 \cdot 10^7$
- Valid_RMSE $\approx 9\,725$
- Valid_MAE $\approx 3\,888$
- Valid_R² ≈ 0.90

Aceste rezultate sunt mult mai bune decât cele ale modelelor lineare și SVR, ceea ce arată că un ansamblu de arbori reușește să surprindă mult mai bine nelinearitățile și interacțiunile dintre variabile (de exemplu combinații între marcă, vechime, kilometri, motorizare etc.). RandomForest este deja un model foarte puternic și stabil pentru predicția prețului.

4. Gradient Boosting Regressor (loss = squared_error)

Pentru GradientBoostingRegressor cu loss="squared_error", ideea este să construim arbori de decizie secvențial, fiecare nou arbore corectând erorile modelului format din arborii anteriori. Hiper-parametrii variază pe:

- n_estimators – numărul de arbori (iterații de boosting),
- learning_rate – cât de mult contribuie fiecare arbore nou,
- max_depth – complexitatea fiecărui arbore,
- subsample – proporția de date folosită la fiecare arbore,
- max_features – câte trăsături se testează la fiecare split.

Cea mai bună configurație găsită folosește o adâncime mică-medie pentru arbori, un learning_rate moderat și un număr suficient de mare de arbori, ceea ce produce un model expresiv, dar bine regularizat.

Cea mai bună configurație găsită este:

```
subsample = 0.8 max_depth = 3 max_features = 0.5 learning_rate = 0.1
```

n_estimators este ales din (200, 300, 400, 600) și combinat cu cei de mai sus

Adică arbori mici (adâncime 3), cu un learning_rate relativ agresiv (0.1), un număr suficient de mare de arbori și folosirea a 80% din date la fiecare iterare (stochastic gradient boosting), plus 50% din trăsături la fiecare split.

Pe setul de validare, acest model obține:

- Valid_MSE $\approx 8.21 \cdot 10^7$
- Valid_RMSE $\approx 9\,061$
- Valid_MAE $\approx 3\,715$
- Valid_R² ≈ 0.91

Gradient Boosting cu loss pătratic este astfel cel mai bun model pe setul de validare, cu cel mai mic MSE și cel mai mare R². Comparativ cu RandomForest, reușește să mai reducă puțin eroarea (diferență de câteva sute de euro la RMSE), confirmând că boosting-ul de arbori este foarte potrivit pentru structura complexă a datelor Autovit.

5. Gradient Boosting Regressor (loss = quantile)

În continuare, am folosit GradientBoostingRegressor și cu loss="quantile", pentru a estima cuantile ale distribuției prețului, nu doar valoarea medie. Am antrenat trei modele separate, pentru:

- alpha = 0.10 – cuantila 10% (scenariu pesimist, preț minim așteptat);
- alpha = 0.50 – cuantila 50% (mediana);
- alpha = 0.90 – cuantila 90% (scenariu optimist, preț maxim așteptat).

Hiper-parametrii n_estimators, learning_rate, max_depth, subsample, max_features au fost căutați în intervale similare cu cele din cazul loss="squared_error". Pentru alpha = 0.5 (mediana), configurația optimă este foarte apropiată de cea a modelului cu loss pătratic: arbori mici/medii, learning_rate moderat, număr suficient de mare de arbori.

Hiper-parametrii cei mai buni găsiți pentru fiecare cuantila sunt:

```
<> alpha = 0.10 (Q10, scenariu pesimist) subsample = 0.6 n_estimators = 400 max_depth = 4 learning_rate = 0.1 max_features = None
```

```
<> α = 0.50 (mediană) subsample = 0.8 n_estimators = 300 max_depth = 4 learning_rate = 0.1 max_features = None
```

```
<> α = 0.90 (Q90, scenariu optimist) subsample = 0.8 n_estimators = 300 max_depth = 4 learning_rate = 0.05 max_features = 0.5
```

Pe setul de validare, GBR quantile (median) obține:

- Valid_MSE $\approx 1.05 \cdot 10^8$
- Valid_RMSE $\approx 10\,243$
- Valid_MAE $\approx 3\,722$
- Valid_R² ≈ 0.89

Performanța punctuală este puțin mai slabă decât la GBR cu loss="squared_error", dar apropiată de RandomForest. În schimb, împreună cu modelele pentru alpha = 0.1 și alpha = 0.9, putem construi banda [Q10, Q90] pentru preț, cu un coverage de aproximativ 78.8% pe setul de validare, deci un interval de predicție rezonabil pentru analiza incertitudinii.

6. Quantile Regressor

În final, am folosit și QuantileRegressor pentru a estima direct cuantilele distribuției preț, într-un model liniar. Au fost antrenate trei modele, pentru:

- quantile = 0.1 – limita inferioară,
- quantile = 0.5 – mediana,
- quantile = 0.9 – limita superioară.

Hiper-parametrul principal este alpha, care controlează regularizarea L2 (mărimea coeficienților). Căutarea a selectat valori foarte mici de alpha (0 sau 0.0001), ceea ce indică faptul că, pentru a surprinde cât de cât relația dintre trăsături și preț, modelul liniar are nevoie de flexibilitate (regularizare minimă).

Hiper-parametrul principal este alpha (regularizare L2). Căutarea a dat:

```
τ = 0.1 → alpha = 0.0 τ = 0.5 → alpha = 0.0001 τ = 0.9 → alpha = 0.05
```

Toate modelele folosesc fit_intercept=True și solver="highs".

Pe setul de validare, QuantileRegressor (median) obține:

- Valid_MSE $\approx 3.21 \cdot 10^8$
- Valid_RMSE $\approx 17\,912$
- Valid_MAE $\approx 6\,882$
- Valid_R² ≈ 0.65

Ca model punctual este clar mai slab decât arborii, dar punctul său forte este altul: împreună cu modelele pe cuantile pentru $\tau = 0.1$ și $\tau = 0.9$, permite construirea unei benzi de predicție [Q10, Q90], cu un coverage de aproximativ 81.6% pe setul de validare. Față de GradientBoostingRegressor cu loss="quantile", QuantileRegressor este un model liniar, mai ușor de interpretat (coeficienți direct interpretabili), dar cu putere de expresie mai redusă.

Față de GradientBoostingRegressor cu loss="quantile", QuantileRegressor are putere de expresie mai redusă (este liniar) și depinde mai mult de calitatea feature engineering-ului, dar câștigă la capitolul simplitate și interpretabilitate.

Tabelul de mai jos sintetizează rezultatele pentru toate modelele testate:

model	CV_MSE	Valid_MSE	Valid_MAE	Valid_RMSE	Valid_R2
GBR (squared)	1.052242e+08	8.210861e+07	3714.55298	9061.38005	0.911097
RandomForest	1.110635e+08	9.457155e+07	3888.48476	9724.79052	0.897602
GBR quantile (median)	1.178459e+08	1.049127e+08	3722.27173	10242.69114	0.886405
LinearRegression	2.573440e+08	2.372652e+08	8094.82411	15403.41623	0.743100
QuantileRegressor (median)	3.186688e+08	3.208257e+08	6881.89782	17911.60810	0.652625
SVR	3.223653e+08	3.242758e+08	6946.35184	18007.65812	0.648890

Per ansamblu, tabelul confirmă clar că modelele pe arbori (Gradient Boosting și RandomForest) sunt de departe cele mai potrivite pentru predicția prețului mașinilor, în timp ce modelele liniare și SVR rămân doar baseline-uri de comparație.

Gradient Boosting Regressor (loss = squared_error) este modelul cu cele mai bune scoruri pe validare (MSE și RMSE cele mai mici, R² ≈ 0.91).

RandomForest este foarte aproape ca performanță (R² ≈ 0.90) și oferă o alternativă robustă, mai simplu de interpretat la nivel de importanță a trăsăturilor.

GBR quantile (median) este puțin mai slab ca RMSE, dar foarte util când vrem și cuantile (intervale de preț).

LinearRegression, SVR și QuantileRegressor sunt net inferioare ca performanță, confirmând că relația dintre trăsături și preț este puternic neliniară și cel mai bine captată de ansamblurile de arbori.

În experimentele pentru Autovit am folosit 5-Fold Cross-Validation pentru a evalua și compara modelele într-un mod cât mai robust. Ideea este să nu ne bazăm pe o singură împărțire train/valid, care poate fi întâmplător favorabilă sau defavorabilă unui model, ci să împărțim întregul set de date în 5 „fold-uri” aproximativ egale, să antrenăm modelul de 5 ori, de fiecare dată pe 4 fold-uri și să îl testăm pe fold-ul rămas. În acest fel, fiecare observație ajunge cel puțin o dată în setul de validare, iar scorurile (MSE, RMSE, R^2) obținute pe cele 5 runde sunt apoi mediate pentru a obține o estimare mai stabilă a performanței. Cross-validation-ul este integrat direct în RandomizedSearchCV, astfel încât pentru fiecare combinație de hiper-parametri scorul raportat este deja media pe cele 5 fold-uri, ceea ce permite o selecție mai corectă a configurației optime pentru fiecare algoritm.

Partea 4: Evaluarea finala

Pentru evaluarea finală a modelelor pe Autovit am folosit un set separat, val_cars_listings.csv. Din acest fișier am păstrat exact aceleași coloane folosite la antrenare (used_cols), am separat feature-urile de target și am obținut:

Val final shape: (4746, 31) - adică 4746 de anunțuri și 31 de trăsături de intrare (după preprocesare și selecție).

Modelele nu au fost evaluate direct în forma lor din faza de validare, ci au fost reantrenate de la zero pe toate datele disponibile de train, adică pe concatenarea $X_{train} + X_{valid}$ și $y_{train} + y_{valid}$ (X_{full} , y_{full}). Pentru fiecare model considerat „cel mai bun” la pasul anterior (stocat în best_models), funcția compute_eval face următorii pași: clonează estimatorul (pentru a nu strica instanța originală), îl antrenează pe X_{full} , y_{full} , apoi prezice prețurile pentru X_{val_final} și calculează metricile Eval_MSE, Eval_MAE, Eval_RMSE și Eval_R2. Fiecare rând cu rezultate este adăugat în eval_rows, iar la final acestea sunt puse într-un DataFrame (eval_df) indexat după numele modelului și sortat după Eval_MSE.

Pentru modelele de cuantile bazate pe Gradient Boosting, codul antrenează separat cei trei estimatori pentru $\alpha = 0.10, 0.50$ și 0.90 , salvează predicțiile pe X_{val_final} (quantile_eval_preds) și calculează coverage-ul benzii [Q10, Q90]: proporția de anunțuri pentru care prețul real se află între cele două cuantile. Apoi este evaluat explicit și modelul de mediană „GBR quantile (median)”, printr-un apel la compute_eval. Un proces similar se aplică pentru QuantileRegressor: se reantrenează cele trei modele pentru $\tau = 0.10, 0.50, 0.90$, se calculează banda [Q10, Q90] și coverage-ul ei pe val, apoi se evaluează și varianta de mediană „QuantileRegressor (median)”.

Tabelul final de evaluare este:

model	Eval_MSE	Eval_MAE	Eval_RMSE	Eval_R2
RandomForest	6.447498e+07	3741.331166	8029.631254	0.924916
GBR (squared)	6.580498e+07	3833.239254	8112.026727	0.923367
GBR quantile (median)	7.777468e+07	3621.698988	8818.995342	0.909427
LinearRegression	2.235660e+08	8071.407043	14952.123903	0.739646
QuantileRegressor (median)	2.944756e+08	6904.906294	17160.290102	0.657068
SVR	2.977258e+08	6953.716516	17254.733005	0.653283

Se observă clar că RandomForest este modelul cu cele mai bune rezultate pe setul de validare final: are cel mai mic Eval_MSE, un Eval_RMSE ≈ 8029 EUR, Eval_MAE ≈ 3741 EUR și cel mai mare Eval_R2 ≈ 0.925 , ceea ce înseamnă că explică peste 92% din variația prețului pe date complet nevăzute. Imediat după el se află GradientBoostingRegressor (loss = squared_error), cu Eval_RMSE ≈ 8112 EUR și Eval_R2 ≈ 0.923 , foarte apropiat ca performanță. Modelul GBR quantile (median) are un RMSE ceva mai mare și $R^2 \approx 0.91$, dar MAE chiar ușor mai mic, fiind în același timp baza pentru banda [Q10, Q90] construită cu modelele de Q10 și Q90. Modelele LinearRegression, SVR și QuantileRegressor (median) au erori mult mai mari și R^2 semnificativ mai mici, confirmând că relația dintre trăsături și pret este puternic neliniară și este surprinsă corect doar de ansamblurile de arbori.

După determinarea celui mai bun model (best_eval_label), codul selectează estimatorul corespunzător (în acest caz RandomForest), îl clonează, îl reantrenează pe întregul X_{full} , y_{full} și generează predicțiile finale pentru setul de validare (preds_val). Acestea sunt salvate într-un fișier pred_autovit_best_model.csv, care conține, pentru fiecare anunț din val_cars_listings, coloanele id, pret_real și pret_pred. În concluzie, pe baza acestor rezultate, RandomForest este ales ca model final pentru predicția punctuală a prețului, iar modelele de cuantile (GBR quantile și QuantileRegressor) sunt folosite complementar pentru a descrie incertitudinea prin benzi de predicție [Q10, Q90].